



Bioinformatics Programme

Technical University of Munich

Ludwig-Maximilians-Universität München

Master's Thesis in Bioinformatics

Representing Phased Haplotypes as a Pangenome

Leon Rauschning





Bioinformatics Programme

Technical University of Munich

Ludwig-Maximilians-Universität München

Master's Thesis in Bioinformatics

Representing Phased Haplotypes as a Pangenome

Repräsentation haplotypaufgelöster Genome als Pangenom

Author:	Leon Rauschning
Supervisor:	Dr. Craig Dent
Advisor:	Prof. Korbinian Schneeberger, Faculty of Biology, LMU
	Prof. Mathias Wilhelm, School of Life Sciences, TUM
Submitted:	15/04/2026

I confirm that this Master's thesis is my own work and that
I have documented all sources and materials used.

15th of April 2026, Munich

Abstract

Recent advances in Ultralong read technology have made single-platform phased genome assembly possible in humans. What used to require a major consortium for a single sample can now be done by a single lab in dozens. However, this has not yet been demonstrated in more complex, polyploid genomes. It is unclear if current sequencing technologies alone can produce phased assemblies in such genomes, or indeed what factors decide if a genome can be phased by a given dataset.

But once a phased genome is assembled, further challenges await: genomes need to be compared, annotated or otherwise analysed to gain biological knowledge. Any analysis requires a suitable representation. Representing modern datasets of many high-quality genomes is an open problem; arguably, representing the genomes of a large dataset is now harder than assembling them.

In this thesis, I present contributions to both of these major open challenges in genomics. First, to address the input requirements of phased assembly, a model of read-based phasing is introduced. The model is used to derive requirements for a read dataset to contain sufficient information to phase a genome of arbitrary ploidy. This proves that read-based phasing can be feasible even in polyploids, but current sequencing technology is still unable to fully phase very complex genomes such as those of autotetraploid potato cultivars.

Second, a graph genome representation termed the Multisyntenic Pangenome Graph is presented. It represents the input sequences in a graph structure based on an extension of the concept of synteny to multiple genomes. A node in the graph represents sequences in multiple haplotypes that are syntenic to each other, i.e. do not just align, but also form part of a colinear backbone. Respecting this stricter criterion during graph induction ensures the graph has a bubbles-on-a-string architecture with regular bottlenecks induced by conserved genome structure. It can be proven to not contain any cycles or nonlocal edges, demonstrating the potential utility of this representation.

These two contributions lay the groundwork for producing and representing large datasets of phased assemblies in an interpretable manner, even in challenging genomes.

Introduction

Recent progress in genome sequencing and assembly technology has made full genome assemblies more and more accessible, to the point where it is now feasible to obtain assemblies for large sample populations in many species[1, 2]. Making this data usable has, however, been a significant challenge so far. Genome assemblies are a large and heterogeneous kind of data. Making the information contained within them accessible is far from trivial, and requires carefully designed computational tools to efficiently handle large datasets and work well in HPC systems.

The matter of how to represent genome assembly datasets is absolutely crucial to any attempt to leverage them to gain understanding. A well-designed representation is required to design efficient algorithms, but any representation will also limit the questions we can ask of a dataset. Despite several recent proposals and a slowly maturing software ecosystem[3, 4], I will argue that the representation of today's genome datasets is an unsolved question still requiring considerable conceptual contributions. To inform this discussion and provide background to the approach proposed in this thesis, the two sections below give a history of how genomics got to where it is now and what kind of datasets are possible given the current state of the art. In the section after that, I will introduce recent proposals for pangenome representations with a focus on their conceptual underbellies and limitations. To close the introduction, I will introduce the core concepts of two softwares used in the genome representation proposed in this thesis. It is my hope that this introduction provides context and a motivation for working on genome representations, particularly for readers that consider themselves more as users than developers of genomic technology – these are not »just« some file formats, but the foundation of any work in the field; choices or tradeoffs made in their design and implementation percolate through to tools and ultimately analyses. As formats and representations tend to last a long time[1], they should be designed with care and conceptually motivated, as any limitation will restrict the field long into the future.

Historical Background to Modern Genomics

The first recognition of the genome as a concept dates to Gregor Mendels work in peas[5, 6]. He represented the genome of his peas as combinations of upper/lowercase letters indicating different alleles. Given the lack of any genomic technologies, the genetic identity of a given pea could only be determined by its phenotype or the phenotypes of its offspring in a controlled cross. Mendel's genomes also lacked any internal structure; gametes were viewed as produced from independently assorted alleles of the parental genotypes. The genome as such was a passive container of alleles, and not conceptualized further.
After the rediscovery of Mendel's work in the early 20th century, the discovery of chromosomes as the bearer of heredity information and further crossing experiments performed among others by Thomas Hunt Morgan's laboratory in *Drosophila melanogaster* revealed the genome to consist of chromosomes

[1] Both FASTA and PDB format are nowadays older than many of its users.

with a linear structure[7, 6]. Through careful crossing, chromosomes were mapped, again determining the genotype solely from the phenotype of the organism and its offspring. These genetic maps charted genes with observable phenotypes along the chromosomes, and revealed a shuffling process known as recombination, acting by discrete crossover events with a determinable likelihood at each position along the genetic map.
The genome had thus evolved from a few unordered letters to a set of linear maps, whose cellular component (if not molecular identity) was known to contemporary researchers. Like Mendel, however, genes along the genome were thought of and named for their observable, segregating phenotypes, and the absence of any genomic technologies made genotyping a slow and labourious process.
Later, microscopy of chromosomes revealed a banded structure, which work in maize by Barbara McClintock and in *Drosophila spp.* by Theodosius Dobzhansky showed to sometimes vary across individuals, initially without giving rise to observable macroscopic phenotypes[8]. The discovery of these so-called »cryptic polymorphisms« was the first time a genotype had been determined by other means than phenotypic observation. Cytogenetics can thus be termed the first genomic technology; in fact, through careful observation of the band structure, these polymorphisms could be identified as inverted or copy-pasted segments along the chromosomes, hinting at the mechanism of their formation – later giving rise to McClintock's discovery of transposable elements[9].
Increasingly, the study of the genome became emancipated from the reliance of clearly segregating phenotypes for every genetic polymorphism, and the discovery of rearrangements revealed that genomes in a species could vary beyond the single-gene level.

The discovery first of DNA as the molecule carrying genetic information and later of its 3D structure in the 1940s and 1950s by Avery[10] and Franklin, Crick and Watson[11] provided a vision to describe the genome at a molecular level. More molecular studies followed like the decoding of the genetic code and the discovery of promoters and transcription factor binding sites, providing some understanding on a basepair level[12, 13]. This opened the door to a radically different way of conceptualizing the genome – not as an ordered map of segregating phenotypes or bands under a microscope, but as a string of bases along a phosphate backbone acting by biochemical mechanisms. As DNA became accessible by cloning, PCR and eventually early sequencing methods, databases of DNA sequences started to emerge. These would contain small fragments of the genome, often genes of particular interest, promoter or plasmid sequences. The sequences were stored as letters, but unlike Mendel's early use of letters to indicate alleles, the letters now indicated the bases **A**denine, **C**ytosine, **G**uanine and **T**hymine and were ordered by their position in the genome, typically starting from the $5'$ and ending at the $3'$ end. Early databases were still printed on paper and distributed as books by mail to university libraries, but they soon embraced early mainframe computers as they became available at universities. Databases used a variety of file formats at the time – often based on previous print conventions –, but the format used by the popular FASTP and FASTA programs came to dominate[14, 15]. This format represents sequence in an `ACGT`-encoding with nonsignificant linebreaks to fit on early computer screens, along with header lines containing the sequence identifier to differentiate different sequences in the same file. Additional metadata can also be added to the header line, and gaps inserted indicating individual unknown bases or unresolved breaks in the sequence. FASTA's format and variants based on it are still in common use today for a variety of purposes. In particular, it has become the de facto standard for representing reference genome assemblies after being chosen by major reference genome projects for distributing their references.

Increasing throughput of Sanger sequencing opened up the possibility of obtaining the base sequence of entire genomes. After early success in viral and bacterial genomes in the early 1990s, this culminated in large, well-funded global consortia preparing reference genomes for a number of eukaryotic species in the late 90s and early 00s, most notably *A. thaliana*, mouse and human[16, 17, 18, 19]. The ambition to sequence not just individual genes of interest, but the entire genome can be viewed as a revival

of Hunt Morgan's concept of a per-species genetic map. The genome would not just be a scientific result in and of itself, but serve also as a community resource and to avoid duplicating sequencing and mapping efforts by individual groups. Technology and standards development, along with outreach to the scientific community and general public, formed a major part of the Human Genome Project and contributed to its lasting legacy. Altogether, this wave of genome projects aimed to establish a new mode of thinking about genomes and more broadly biology, »ushering in the post-genomic era« in the language of the time[20, 21].

With the reference genome deciphered for several model species, genomicists were now able to compare samples to the reference genome – both conceptually to provide a language for describing genomes in terms of their differences to the reference, and by using the reference to develop microarrays that could detect known differences in a sample at a very low cost per sample. This powered the HapMap project immediately after the human reference had been assembled, aided by the long linkage blocks and low overall variation in the human genome[22]. Later, sequencing costs declined sharply with the advent of early next-generation sequencing technologies. Especially in the beginning, these produced very short and error-prone untargeted reads that were not directly useful on their own. However, the reads could be aligned to the reference genome, recovering the position they were sampled from. Many classic read alignment tools were developed in this period, based on earlier approaches for searching for genes in reference genomes and databases[23, 24]. Small differences in sequence between sample and reference could then be determined from these aligned reads using statistical models[23, 25]. A key advantage over microarrays was the ability for *de novo* variant discovery and to avoid the need for labourious microarray design and manufacturing, particularly in non-model organisms. At least for single-nucleotide variants or small indels in non-repetitive regions, next-generation sequencing allowed scalable and comprehensive genotyping. This drove a wave of projects sequencing ever larger samples, such as the 1000 Genome Project in human and the 1001 Genomes Project in *A. thaliana*[26, 27]. The motivation for these projects was increasingly shifting away from a technology development and community resource perspective towards association studies and medical applications (in humans).

In order to efficiently compute associations between genotype and phenotype, genotypes of multiple samples have to be aggregated into a single, numerical matrix representation[28]. To store the genotypes collected in large-scale sequencing efforts, the Variant Call Format was developed by the 1000 Genomes Project[27]. VCF closely follows this matrix representation, and while updates over the years have added ways to encode larger variations into the standard, its representation remains centred on cataloguing single-nucleotide polymorphisms in many samples[29].
This represented a step away from genome assembly, going from the deciphering of the exact (if at the time usually not complete) base sequence to a list of genotypes at segregating sites, analogous to Mendel's genotypes but at much higher scale and without requiring directly observable phenotypes for every variant.

In the late 2010s, long read sequencing technology became available. Initially, these technologies suffered from high error rates and were costly to use[30, 31], which made them less interesting for large cohort sequencing projects. However, they would prove to spur something of a renaissance in *de novo* genome assembly, particularly in challenging repeat-rich genomes[32, 31]. Using long reads and suitable assembly algorithms, much more contiguous genome assemblies could be produced, requiring less or even no scaffolding[31]. For a more in-depth discussion of long-read sequencing technologies and assembly algorithms, see the next section. While early long-read assemblies contained many small-scale errors due to high error rates in the underlying sequencing technology, »polishing« algorithms using short reads from the same sample for error-correction quickly addressed this issue [2][33, 34]. The availability of long-read sequencing inspired the ongoing Earth Biogenome Project and

[2]Recent decreases in error rates and error correction algorithms have made this unnecessary. Polishing can introduce

Vertebrate Genome Project's efforts to provide a reference assembly for every known eukaryotic/vertebrate species [3][35, 36].
Using long reads, the T2T consortium could for the first time decipher the complete sequence of a human autosomal genome (and, separately, the human Y chromosome) in 2022/23[37, 38]. While this initial assembly was a highly manual and very cost-intensive approach [4], the lessons learned along with further improvements in long-read technology allowed the development of assembly algorithms and sequencing standards making the economical and reliable production of chromosome-scale assemblies in human and many other species possible.

Echoing the start of the HapMap project after the conclusion of the Human Genome Project, after this initial assembly interest has turned to cataloguing diversity and providing a community resource, an effort spearheaded by the Human Pangenome Reference Consortium (HPRC). Instead of a haplotype- or small variant-focused effort, the consortium aims to assemble the genomes of a representative sample of humanity[39]. But the goals of the HPRC extend beyond cataloguing variation. It also aims to provide as a community resource a reference based not on a single well-assembled haplotype, but aggregating genome sequence across the sample population. This is claimed to improve downstream analyses, particularly by eliminating reference bias in genotyping[40, 39]. To make any new reference usable, a standard file format and software ecosystem around it are required. Developing these standards is a stated aim of the human pangenome project[39, 41].
Pangenome projects are not restricted to humans, but there are similar efforts in all major model organisms and many species of agricultural or ecological interest [42, 43, 2, 44, 45].

After going into more details about the current state of the art in sequencing and genome assembly in the following section, I will survey some proposed approaches for a pangenomic reference in the section thereafter. I will conclude that while a software ecosystem is emerging, there is no consensus on the underlying concepts of either a genome representation or a reference, or what properties would be desirable for them to have. In fact, even the language used to discuss this subject is not always used in a consistent way in the literature.

errors by over-correcting, particularly in repetitive regions or for heterozygous sites.

[3]With the Earth Biogenome project in particular acting as a common roof for many other clade-specific sequencing projects.

[4]Not entirely unlike the initial Human Genome Project.

Assembling Genomes

In this section, I will give a brief introduction into the process used to assemble a genome, highlighting the current state of the art. In particular, I will focus on long read-based *de novo* phased assembly, as this offers the most complete characterization of the genome and has recently become economically feasible in larger samples, including in species with fairly complex genomes.

DNA Sequencing Requirements

As genomes are far too large to be sequenced in one piece, they are fragmented and the base sequence of some of these fragments reconstructed; these sequenced fragments are known as reads. The problem of reconstructing the sequence of the entire genome from the sequence of the sampled reads is known as genome assembly, and is often compared to piecing together a jigsaw puzzle. However, there are a few key differences between genome assembly and solving a puzzle; chiefly, sequencing reads vary in length and may contain sequencing errors or PCR artifacts, and genomes often contain highly repetitive regions. As an additional difference, reads are typically sampled randomly from the genome, an approach known as shotgun sequencing for historical reasons[46, 47]. Unlike the regular coverage of a picture by its jigsaw pieces, this requires accounting for significant stochasticity in the input dataset.
We can already see a minimum requirement for a dataset of sequenced reads to be used to successfully assemble a genome; there need to be sufficient reads at every position to correct any sequencing errors present in individual reads, lest they be propagated to the final genome assembly. As during sequencing certain motifs are far more error prone than others, this means substantially more sequencing than the genome length is required. The amount of sequencing data in a dataset is referred to as the coverage or (sequencing) depth; it is typically expressed as a multiple of the (haploid) genome size, e.g. 10x or 20x coverage.

In practice, this is usually a minor problem, as it can be avoided by simply sequencing to a higher depth – but read errors add significant complexity to genome assembly techniques, as explained in the next subsection.
A more stubborn problem is presented by repetitive or low-complexity regions. These make reconstruction challenging by causing spurious similarity between reads, obscuring the real similarity signal – in the puzzle analogy, this corresponds to the blue sky or dense foliage being the most challenging regions of the jigsaw puzzle to assemble.
Longer reads can help resolve repetitive sequences in two ways. First, a sufficiently long read can span the entire region containing the repeat, allowing direct reconstruction. Second, most repetitive regions are not exact repeats; like other regions in the genome, they accumulate mutations as they evolve. As a read gets longer, it can cover more of these imperfections that can be used to resolve the repetitive region if the sequencing error rate is low enough to confidently identify them.

It comes as little surprise then that recent developments in sequencing technology have primarily focused on increasing read length and decreasing error rate in long-read sequencing. Read length as well as error rate are primarily driven by sequencing technology, but steps required to extract DNA from a sample and prepare it for sequencing may also limit achievable read length and coverage, especially in small or difficult samples[48, 49].

Long Read Sequencing

As outlined in the Historical Background section, modern genome assembly relies on chiefly on long reads, typically defined as sequencing technologies allowing fragments of at least 10 kbp to be assembled. In contrast, earlier short read approaches could sequence no more than about 400 bp, or about 1 kbp when considering paired ends of a read with an unsequenced gap in between.
Today, two companies dominate long read sequencing: Oxford Nanopore Technologies (ONT), founded in 2005, and Pacific Biosciences (PacBio), founded as Nanofluidics Inc. in 2004. The two companies develop two completely different sequencing technologies: PacBio's approach is based on sequencing by synthesis, similar to dominant short read techologies. A single strand of the sample DNA is introduced along with labelled nucleotides to the flowcell serving as a consumable reaction vessel. The flowcell is studded with individual DNA polymerases, to which the DNA can bind. As the polymerase synthesizes the complementary strand to the input DNA, the label is cleaved from the incorporated nucleotide. This results in a fluorescence signal, which is read out by the sequencer to reconstruct the read's base sequence. This sequencing technology is sometimes also known as Single-Molecule, Real-Time (SMRT) Sequencing[50].
In contrast, ONT's platform relies on an eponymous nanopore. The biotechnologically produced nanopore is situated in a membrane separating two small cells, across which a voltage is applied. This induces an ionic current that flows through the pore, and scales with the pore's resistance and the applied voltage[51]. The sample DNA is attached to a motor protein, and fed through the pore. The resistance of the pore and thus the measured current varies based on which nucleotides are currently in the narrowest part of the pore. From this signal, the read sequence is reconstructed using a Machine Learning model[51, 30].

Modern developments in long-read sequencing technology arguably started with PacBio's announcement of HiFi sequencing in 2018, providing for the first time long reads with a low error rate[52]. This was made possible by preparing a sequencing library containing circular instead of linear molecules of DNA with an adapter sequence, allowing the same read to be sequenced multiple times and a consensus call to be made.
In 2023, PacBio released the Revio sequencing platform, drastically improving throughput and driving down sequencing costs both on release and in subsequent improvements to the platform[53, 54].

At the same time, ONT's sequencing platform received multiple improvements, helping to reduce its error rate and increase throughput[30]. Most notable among these are the release of the R10 pore chemistry in 2020 and the Dorado basecaller in 2022[55, 56, 57]. In 2024, HERRO was released, demonstrating a substantial reduction in error rate for ONT reads by all-against-all alignment and Machine Learning-aided consensus calling[58]. This technique was quickly adopted both by ONT's Dorado basecaller (called Dorado correct) and hifiasm.

While Oxford Nanopore Technology's sequencing platform had always been capable of sequencing very long reads[59], more stable pores and library preparation protocols optimized for ultralong reads made longer and longer read lengths achievable in practice. Individual megabase-scale reads are now regularly observed in mammalian genomes, with the longest reported read announced by Oxford Nanopore to be 4.3 Mbp, or more than twice the size of the *Haemophilus influenzae* genome, the first non-viral genome ever assembled[60, 61].

To summarize, both ONT and PacBio sell state-of-the-art long read sequencing platforms. While ONT can produce dramatically longer reads with the right library prep, PacBio still has a lower error rate, though this gap has narrowed substantially with the introduction of read error correction. While PacBio sequencing machines are bulky and expensive, ONT sequencers are small and more affordable, though consumable costs are higher and throughput is lower per machine. In the next subsection, I will discuss how these modern sequencing technologies are used to assemble a genome.

Assembly Algorithms

Sequencing a genome is is not enough to obtain a genomic sequence; the genome assembly puzzle still needs to be solved. Even for the smallest eukaryotic genomes, sequencing datasets are far too large for humans to process, making computer programs the mainstay for genome assembly from the very beginning, though human judgement is still often required for quality control, setting appropriate parameters and scaffolding. Genome assembly is a complex computational problem [5], meaning that heuristic algorithms are required to make the problem tractable. Early approaches used simple greedy heuristics joining maximally overlapping reads, but these often caused misjoins between different sequences containing the same transposable elements[64, 65, 66]. Almost all later assembly softwares use one of two algorithmic paradigms, both based on a graph representation of the sequencing reads [6].

De Bruijn-graph based assemblers split reads into overlapping fragments, termed k-mers. A k-mer found in the reads is represented in the de Bruijn graph as an edge. If they are not already in the graph, the two k-mers of length $k-1$ obtained by removing the first and last letter from the original k-mer are added to the graph as (labelled) nodes. After creating the nodes if necessary, an edge corresponding to the read k-meris added between them[68, 63]. The genome has to contain all k-mers found in the reads; as such, it corresponds to a path traversing all edges in the graph, also known as an Eulerian path. The Eulerian path in a graph can be found efficiently, but exact repeats of size greater than k will be collapsed, and sequencing errors need to be identified and corrected prior to graph construction. Most de Bruijn-graph based assemblers include additional heuristics to address these limitations.

The other approach is referred to as Overlap-Layout-Consensus (OLC)[69]. In the graph representation used, entire reads are represented as nodes. An edge is added between reads with overlapping sequence at their ends – this is known as the overlap step. This initial representation, also referred to as an overlap graph, is processed in a variety of ways, notably by compressing unbranching paths, removing transitive edges and reads entirely contained within another read. From this simplified graph representation, known as a contig or assembly graph, the genome is reconstructed in the layout step by searching for an optimal path covering all nodes (equivalent to finding a linear layout of the graph, hence the name). Finally, reads are aligned back to the path to correct sequencing errors in the final consensus step.

The layout step in particular is a very hard algorithmic problem, requiring aggressive processing of the graph representation and heuristic approaches in order to be tractable. Finding all read overlaps is also a computationally demanding step, although it can be parallelized efficiently.

Remarkably, both of these approaches have been adjusted to successive generations of sequencing technologies over the years, and neither has proven to definitely outperform the other as of now.

Currently, two assemblers are used for state of the art assembly. Verkko is based on a de Bruijn graph approach, and hifiasm uses the OLC paradigm[70, 71]. Both of these assemblers have similar specialized procedures integrating ONT ultralong reads to resolve difficult regions and improve assembly contiguity, as well as preprocessing steps performing read error correction[72, 73]. To integrate ultralong reads, they are aligned to the initial unitig graph (hifiasm) or a compressed de Bruijn graph (verkko). These alignments are then used to refine the graph, by adding additional edges (both) and duplicating nodes to resolve ambiguous paths (only hifiasm). This allows both assemblers to utilize ultralong ONT reads to similar effect, boosting contiguity and resolving repetitive regions inaccessible to shorter reads. Though hifiasm tends to be faster and easier to use as a single command-line tool compared to verkkos architecture as a pipeline chaining many steps, both assemblers produce

[5] Most formulations fall within the complexity class of $\mathcal{NP}$-complete problems, with the notable exception of certain Eulerian path approaches that are polynomially bounded by the number of k-mers in a genome[62, 63].

[6] One recently developed linear-time algorithm presents a notable exception, see [67].

assemblies of similar quality. Though they differ in their underlying algorithmic paradigm, this is not too surprising given that recent performance improvements have largely come from the integration of ultralong ONT reads and read error correction before assembly, steps that both assemblers implement in a similar manner.

Phased Assembly

So far, we have seen how the genome of an organism can be determined using long-read sequencing technology and genome assembly algorithms. However, most eukaryotes do not have a single genome to reconstruct; cells often carry multiple copies of the genome, referred to as haplotypes. Variation on the same haplotype is inherited together, unless recombination breaks it up; this has given haplotypes the moniker »the unit of heredity«. As recombination switches genomic sequence between different haplotypes in the same organism, it provides an evolutionary force preventing different haplotypes of a population from becoming too divergent from another[74, 75].
The number of copies is known as the ploidy, and is typically thought of as idiosyncratic to a species, but it may also vary by individual, sex or tissue [7] .

Humans and many animals are diploids, having two copies of their genome.
A. thaliana also has two copies of its genome, however it often described as a haplodiploid (rhyming haploid, meaning single-copy), as its generally selfing nature makes these almost identical save for individual *de novo* mutations.
But many organisms have more than two copies of their genome, and are then known as polyploids. This is the case for many crop species, with e.g. strawberries's 8 copies, Cavendish banana's 3 and sugarcane's approximately 10[76]. Sometimes, different haplotypes exhibit preferential pairing during meiosis. These are known as subgenomes and organisms that exhibit prefential pairing are known as allopolyploids, while organisms with equal pairing between all haplotypes are referred to as autopolyploids. Recombination between different subgenomes tends to be suppressed in allopolyploids, though it can still happen to various degrees[77]. In crops, alloploidy is often the result of a (historical) cross between two divergent lines; the classic example is wheat's hexaploid genome, made up of three subgenomes each present in two copies, deriving from different wild diploids[78].

Separately assembling the different haplotypes of a sample is known as phased assembly. This is a formidable challenge, compounded by the ploidy of the sample and the length of regions where different haplotypes share the same sequence – unfortunately, both high ploidy and long identical stretches are common in crops as a result of (in-)breeding and bottlenecks associated with domestication. While in allopolyploids, different subgenomes will generally have divergent haplotypes, in autopolyploids recombination can break these up, making them more challenging to phase.

Traditionally, separating different haplotypes during assembly has required information beyond that present in the reads. A common approach is to sequence the parents of the sample, and use variants unique to one parent to infer the haplotype of reads or contigs. This is known as trio sequencing. A similar approach sequences the offspring or, as recently demonstrated, gametes of an individual[79]. Trio sequencing requires the parents of the sample to be known and accessible, and all of these approaches except gamete sequencing require additional samples to be obtained and processed, besides the need for more sequencing. These approaches also get more complicated the higher the ploidy of the organism: in trio sequencing, one would need to go back one additional generation for every doubling in ploidy, but every generation adds additional recombination events complicating inference in the sample's data.
Both of these approaches rely on haplotypes, as units of heredity, being generally inherited together.

[7] Gametes and certain cancers are the most well known example, but polyploidy is also regularly observed in healthy human hepatocytes. In hymenoptera, the ploidy/fertilization status of the egg determines the sex of the adult.

This effect is also visible in a population, and indeed a haplotype panel derived from a population matching the sample can be used to infer its phasing. As this leverages more indirect relations than that between parent and offspring, it is more likely to miss recombination events, and generally produces short-range phasing information not considered sufficient for high-quality phased assembly.
Proximity ligation data can also be used to infer the phasing of haplotypes, and does not require access to additional samples. This is the approach recommended by the Earth Biogenome Project, though the most common HiC- or OmniC-platforms require additional library and short-read sequencing for every sample[48]. The short reads used by most proximity ligation platforms may not uniquely align to a haplotype, particularly in highly polyploid and repetitive genomes. The recent development of long-read proximity ligation assays may help to address this as they become more widely available. Ideally, one would phase an assembly directly using the sequencing reads; this has been demonstrated in humans by the hifiasm team, using ONT ultralong reads and an assembly algorithm especially designed for using these reads in phasing (now part of hifiasm)[71].

Genome Assembly in 2026

As outlined above, the last few years have seen major developments in DNA sequencing technology, and in the algorithms that use them to assemble genomes. It is now routinely possible to produce chromosome-scale assemblies – that is, assemblies that are largely complete and contiguous[48, 80]. In diploids, these assemblies can usually be phased to a high standard, although phasing in polyploids is still less reliable and often requires manual curation.

As the error rates of ONT sequencing have gotten lower, it has been demonstrated that genomes can be assembled using just their sequencing technology[48, 81]. In fact, one recent preprint demonstrated chromosome-scale phased assembly in humans (as well as long-read based contact map profiling) using only 4 ONT flowcells at an estimated cost of approximately 3000€ per sample [82]. Using only a single platform not only saves costs, but also simplifies planning and library preparation, allowing smaller teams to process more samples. While for humans and other diploids, experience has provided a »recipe« for a high-quality phased assembly that works reliably, for polyploids the process is still more ad hoc, with unclear input requirements and manual curation[48]. In a literature search, no reference to a successful phased assembly of a polyploid genome based soleley on read information could be found. Motivated by this gap in knowledge, I derive a set of requirements for phasing using a reads dataset to be possible in principle in the first section of the Main.

Contemporary Pangenome Representations

The sequencing and assembly approaches described in the last section have made it possible to produce high-quality assemblies for larger and larger samples. Currently, the Human Pangenome Reference Consortium has released phased genome assemblies of 280 samples as of April 2026, with plans to release the data of 350 samples in summer 2026 [83, 84]. Humans have been the main focus of these efforts, but there are also major projects in model organisms, in particular the 1001G+ project in *A. thaliana* [85, 86, 87, 83]. Increasing throughput and falling costs have also meant that recently, medium-sized assembly datasets can be prepared by individual, well-resourced labs without requiring consortia[88, 42, 89, 82].

Gene-centric Pangenomes

Early work on representing datasets of genome assemblies has been done in bacterial genomics, aided by shorter genomes that are easier to assemble[57, 90, 91]. Though the term *pangenome* was first used in the literature to describe genomic variation in a human tumour dataset, it became widespread in the bacterial community in the late 2000s[92, 90]. Bacterial genomes can change dynamically by taking up or losing plasmids and other mobile genetic elements, often shared across different species and providing traits that are adaptive in a certain environment [93, 94]. Reflecting this genomic plasticity and a straightforward interpretation based on gene annotations, gene-centric representations have historically been the most popular approach in bacterial genomes[90]. While there are many kinds of gene-centric pangenome representations, a common lexicon has evolved in the field, differentiating core genes found in every sample's genome and accessory genes exhibiting presence/absence variation[8][90]. This terminology ignores any variants in or changes in the relative ordering of the genes. While gene-based pangenome representations have been used in eukaryotes as well, particularly for deep phylogenetic studies[95, 90], sequence-based representations tend to be more popular. Recently, some sequence-based pangenome graph representations originally developed for eukaryotic genomes have also made inroads into bacterial datasets[96, 97].

Sequence-based Pangenomes

Gene-based representations disregard the vast majority of the eukaryotic genome that does not code for proteins. This misses many variants, both neutral and affecting gene regulation. As significant effort is spent assembling complete genomes, it is desirable to use a representation that can adequately represent all of the genome. All major eukaryotic pangenome projects have therefore focused on sequence-based pangenome representations, which will also form the basis of this thesis. Almost all sequence-oriented pangenome representations use some kind of a graph as the basis for their approach [9]. The sequence graphs used for this echo the representations used in Overlap-Layout-Consensus assembly algorithms (described in more detail in *Genome Assembly* above), with the exception that pangenome graph representations typically disallow overlaps between different nodes. The major reason for using this graph-centred approach have been the availability of a well-established file format and a sequence graph's flexibility, accomodating quite different approaches in a common software ecosystem[40, 90, 3, 4].

Early sequence-based pangenomes were constructed from VCF files obtained by calling variants between all samples and a common reference[4, 98]. However, these representations inherited all of

[8] Occasionally also into cloud genes, used for genes found only in a single or very few samples.

[9] With the notable of exception of GRCh38's additional unplaced contigs for certain regions known to be structurally polymorphic, if the reader accepts this as a pangenome.

VCF's limitations, in particular VCF's inability to represent nested or complex variants and the ambiguouos way structural variants are represented in VCF. Additionally, VCF loses phasing information that cannot always be reconstructed during pangenome graph construction. Fundamentally, VCF's list of differences approach was designed to store small variant genotypes across many samples for association studies, and is not suited to representing complete genomes. More modern approaches directly construct their pangenome representation from the input sequences, avoiding VCF's limitations.

One may ask why the genomes need any kind of representation at all; storing the full sequence of all samples in separate (or even the same) FASTA file would preserve all information, and make obtaining the sequence trivial. This approach is not without precedent; the BLAST database is maintained in a similar way[99]. While this »non-representation« would indeed not lose any information in the sample genomes, it would run into practical and conceptual limitations in most usecases. On the practical side, representing all sequences separately takes a constant amount of space per sample. This increases storage requirements, but more importantly makes using the reference in downstream analyses more memory- and compute-intensive. For example, a resequencing pipeline would need to align every read to >400 haplotypes if it were to switch to the current human pangenome reference represented in this way. This also shows the conceptual limitations of this non-representational approach; what would it mean in the described experiment for a read to align only to one genome, all 400, or to some but not others – particularly if it does not align equally well to all? Treating all samples separately may not remove any feature of the samples, but it fails to represent the information to be gleaned from their aggregation.
In my view, it is still a useful thought experiment to compare proposed representations with the raw, uncollapsed haplotypes. This can highlight advantages of different proposed representations and elucidate limitations, ultimately allowing an informed choice among different representation and providing directions for the design of future representations.
In particular, two features emerge in this consideration. First, the ability to reconstruct the input haplotype sequences from a representation; this property guarantees that the representation does not lose information present in the samples, and ensures that it can be converted to any other pangenome representation via the raw data. Reconstructing the input may be impossible either because the representation discards some genomic sequence from the input haplotypes, or because the representation is ambiguous as to the arrangement of the sequences – i.e. it keeps the pieces, but not the instruction manual describing how to put them back together. Pangenome representations fulfilling this criterion can be conceptualized as a compression or annotation of the input haplotypes.
Second, a pangenome representation should be compact, scaling slower than the raw uncollapsed haplotypes. This serves to make downstream analyses computationally more feasible given the scale of recent datasets, and to avoid excessive redundancy that would otherwise make it difficult to draw conclusions from approaches using the representation. In regards to this consideration, different representations can be thought of as occupying different places along a spectrum of redundant vs. collapsed representations[40]. While the input haplotypes occupy the extreme end of collapsing nothing and preserving all redundancy, an imaginary fully collapsed representation would consist only of the bases `A`, `C`, `G`, `T`, and arguably be even less useful in practice.
Different proposed approaches sit at different points along this spectrum, and it is helpful to bear this in mind when choosing between them and setting their parameters, particularly considering that different analyses may require or prefer different degrees of collapse[100, 101, 89].

Minigraph

The first modern sequence-based pangenome graph toolkit able to directly construct a graph not based on a VCF representation was `minigraph`, released in 2020 by Heng Li[102]. At its core, minigraph is a `minimap`-inspired graph aligner, that is, a software capable of aligning a query sequence to a reference sequence graph. It can be used to construct a pangenome graph by iteratively aligning samples to the graph. Minigraph's alignment step first uses minimizers to map nodes to the query sequence. These are then chained into an alignment respecting the graph's structure. While minigraph's core algorithm does not use the genomic sequence of either the sample or graph except to find minimizers, support for a final base-level alignment step was added in version 0.17, published in 2022[103].
To construct a pangenome graph using the alignment algorithm, a graph is first initialized as a single node containing a reference sequence. Then, all other sequences in the pangenome are aligned to this graph representation. At each step, unaligned regions in the query sequence are added as nodes to the graph, and partially aligning nodes are broken up into multiple nodes. Due to the minimizer-based heuristic approach, the resultant graph can look different depending on the sample ordering, and small variants are not represented in the graph structure. To recover small variants not detected by minigraph's minimizer-based approach, later efforts developed the minigraph-cactus pipeline[100]. These small variants make the graph »spiky«, hence the name.
But even with these small variants added, minigraph's representation cannot be used to reconstruct the input haplotypes, as minigraph does not report sample's path through the graph [104, 105].

The Pangenome Graph Builder (PGGB)

The Pangenome Graph Builder (PGGB) is the second prominent pangenome graph representation toolkit. It was released initially as a shell script-based pipeline in 2021, then ported to a nextflow-based pipeline and published in 2024[106, 107]. It takes a different approch to minigraph, in that the genome representation is based on the concept of a variation graph. A variation graph is a datastructure representing all pairwise alignments of a set of sequences in a graphical manner[108]. While initially formally introduced as the graph representation of a non-deterministic automaton built from the sequences, I will introduce it here in the language of alignment graphs. Consider the input sequences as segments of arbitrary but constant size [10] . A bipartite graph can be constructed with the segments of two input samples as nodes, and an edge between two nodes iff they align to each other. The segments of another sample can now be added, and the graph extended to a tripartite graph by connecting nodes in the previous two samples with aligning nodes in the new sample; this process can be iterated until the graph includes all samples. A segment shared by all or a subset of samples will induce a clique in this graph, that is a fully connected subgraph. A variation graph can be constructed by clique decomposition of this graph; to do this, consider a maximal set of cliques of the alignment graph described above and take this as the nodeset of a new graph. Then project the paths implied by the sample genome's ordered segments into this path, and optionally collapse nodes where they do not diverge.
The approach taken by PGGB to construct a pangenome variation graph is not too different from this characterization of variation graphs: PGGB first maps segment-based homology using an approximate minimizer-based approach. A random subset of the sequences assigned as homologous is then aligned at basepair resolution. Both of these steps are implemented in the WFmash package, combining the MapMash and WaveFront alignment algorithm[109, 110]. Subsequently, an alignment graph is constructed and transitive closure is applied to recover alignments missed by the subsampling performed in the previous step[111, 107]. Fully connected components are collapsed into nodes

[10] Both overlapping and non-overlapping segments yield a similar alignment graph.

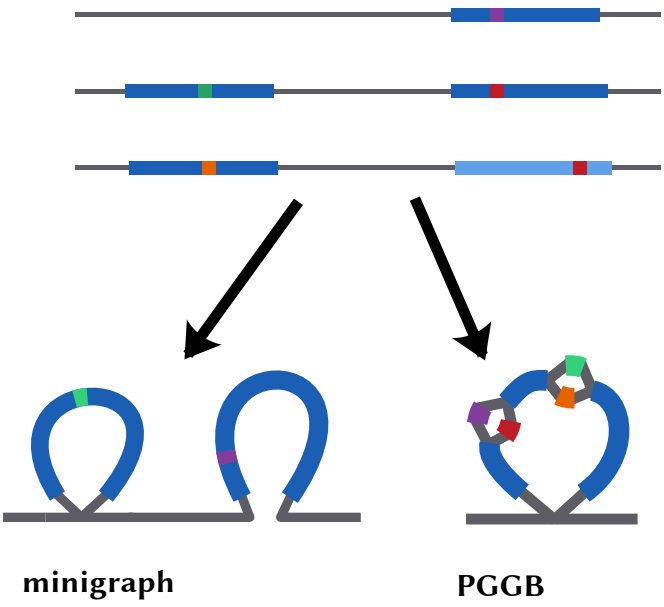


Figure 1: **Illustration of the graph representations produced by minigraph and PGGB.**
Top, the input genomes used to construct the representations. Blue, thick ribbons are identical segments; the light blue ribbon is inverted compared to the other segments. On these ribbons, small variants are drawn in a different colour.
Left, the minigraph representation. Small variants are not represented in minigraph pangenomes, though they are added at a later stage using by the minigrap-cactus pipeline, thus every segment has the allele it is first encountered with marked. Minigraph collapses inversions in the same place into a single node, but does not collapse duplicated segments. Note that the inversion makes this graph cyclic.
Right, the PGGB pangenome graph. PGGB collapses identical sequences, even if they are not adjacent. Small variants are represented as nodes in the graph, and are merged across identical segments. This graph contains multiple cycles, and would also be cyclic if there were no inversion.
Figure adapted from Extended Data Fig. 3 of ref. [39].

in the variation graph-based pangenome graph. This initial representation is postprocessed in multiple steps to remove inconsistencies in the all versus all pairwise alignments and redundant nodes. In this postprocessing step, complex regions of the graph may be realigned using a graph-based multiple sequence alignment. The graph induction is implemented in the seqwish package, while postprocessing uses the SMOOTHXG and GFAfix tools[111, 107, 112].
The PGGB pipeline is quite complex and involves multiple steps with $\mathcal{O}(n^2)$ worst-case complexity, making it very compute intensive although the implementation of base-level alignment subsetting has decreased compute requirementssomewhat. PGGB's nextflow implementation also scales more effectively on HPC systems, decreasing analysis latencies if sufficient computational resources are available.
As PGGB reports (potentially cyclic) paths for all samples, it is possible to reconstruct the input genomes from its representation. Compared to minigraph's representations, PGGB's output graphs tend to be much more collapsed. Though this depends on the exact parameters chosen during the all vs all alignment, in general PGGB collapses all repetitive elements, which can lead to many nonlocal edges in repeat-rich genomes[113]. See Figure 1 for an illustration of how PGGB's representation collapses repeated segments, compared to minigraph's approach.
PGGB graphs also frequently contain cycles, or multiple successive very small nodes that are not straightforward to untangle[113].

An emerging consensus?

The sequence-oriented pangenome ecosystem has seen rapid development in recent years. It has been spearheaded by the HPRC, and has contributed to developing an ecosystem around the GFA1 file format. This includes tools that produce and utilize pangenome graphs in this format, as well as utilities for manipulating them[4, 114, 3].
Originally designed by Heng Li to emit hifiasm's intermediary assembly graph outputs, GFA1 in its original iteration is a graph with nodes as sequences and edges as overlaps. To represent pangenome graphs, a subset sometimes referred to as rGFA1 is used; it disallows overlaps, requiring all edges to report empty overlaps of length 0.
Extensions to the standard have added some pangenome-centric features to the format, such as the ability to report sample paths in the graph. Classical tools for visualizing GFAs have been used to visualize pangenome graphs, but the recent development of ODGI has aided visualization, particularly of PGGB's more densely connected graphs[115, 116, 117, 118, 113].

However, graph genome representations still have some way to go to become well-established in the wider genomics field. In particular, many downstream applications only work with pangenomes produced by either PGGB or Minigraph(-Cactus), although they use the same file format. See also Figure 1 for a comparison of the two representations. Repetitive and structurally complex regions and genomes also still present a considerable challenge, with especially PGGB introducing many nonlocal connections and producing a highly fragmented pangenome graph[113].

Moreover, so far there has not been a clear case of a downstream application using a graph-based representation that would have been impossible or very challenging with either a single linear reference or all uncollapsed haplotypes. The HPRC's first draft release focused on improvements in short read alignment and small variant calling accuracy, and could demonstrate modest performance increases[11][39].
Similarly, some improvements have been demonstrated for short-read structural variant calling, but this approach remains unreliable over all and increasingly seems obsolete given the accessibility of long reads[114]. Genome graphs have been used to reconstruct genomic sequence based on low coverage short- or long-read resequencing experiments[42, 119, 87]. While these approaches manage much more complete profiling of sample genomes than traditional variant-calling based resequencing, they still cannot recover haplotypes not present in the (graph) reference, and struggle with novel recombination events; given the increasing affordability and ease of *de novo* genome assembly from ONT reads, it is unclear if this approach will see community adoption.

Graph representations have been used successfully to perform genetic associations for structural variants in cattle (genotype-phenotype association) and human (eQTL analysis) [120, 87]. This has recovered novel associations, and could present a compelling usecase for graph representations; however, especially in human, phenotypic association studies have been thought to require much larger sample sizes than are currently achievable in graph settings[121], and it is commonly thought that SVs can be reliably tagged using small variants as markers, though this is assertion challenged by the results reported by ref. [87]. Likewise, methods have been developed to use pangenome graph representation for identifying transposable elements, though they typicall still use results from traditional HMM-based approaches [89, 101, 122].

I argue that a major factor holding the field back is a lack of clear conceptual motivation in current approaches besides collapsing similar sequences. This leads to graphs whose elements have unclear biological significance, and which are prone to produce unwieldy structures in complex regions or repetitive genomes. In this thesis, I will propose a different approach to represent sequence-oriented

[11] About 1-2 percentage points for recall, less than 0.1 percentage points for precision in a medical dataset.

pangenomes based on extending the concept of synteny beyond the pairwise comparison of genomes. This representation uses both conceptual advances and software outputs from two softwares introduced in the next section.

SyRI and msyd

The genome representation proposed in this thesis has two main dependencies, the Synteny and Rearrangement Identifier (SyRI) and the Multisynteny Detector (msyd). These dependencies are central to the proposed representation, both in regards software implementation and its conceptual underpinnings. Both SyRI and msyd were previously developed in the same lab; SyRI has been published in ref. [123], while the core msyd algorithm is described and motivated in my Bachelor's Thesis (supervised by Manish Goel, developer of SyRI and now at Illumina). In this section, a brief outline of the inner workings of both methods is given, along with a description of modernizations carried out to make both softwares suitable for this project. This is intended to give readers who are not familiar with either tool the necessary background to understand guarantees on the representation's input that are used in formal arguments later in this thesis, and an understanding of the limitations of these same inputs. The initial version of msyd is described in my Bachelor's Thesis, and in the API documentation.

The Synteny and Rearrangement Identifier (SyRI)

SyRI is a tool written by Manish Goel to compare the genome structure between a reference and query assemblies[123]. Its core insight is that finding *syntenic*, that is structurally conserved, segments is far easier than finding rearrangements, and after all syntenic regions have been found all that is left to do is to annotate the remaining sequence as the right kind of structural rearrangement.
This is reflected in the SyRI algorithm: First, syntenic regions are identified from a whole-genome alignment by selecting a longest path of consistent alignments covering each chromosome. Two alignments are considered consistent if the regions they align occur in the same order in both genomes – intuitively, this can be understood as two lines drawn between the aligning regions not crossing each other (compare also Fig. 2). Finding this longest syntenic path is a complex optimization problem;SyRI approaches it by representing aligned blocks reported by ordinary whole-genome aligners as nodes in a graph. It then finds a longest path in this graph through a divide-and-conquer algorithm, falling back to an exact solver implemented in the `longestrunsubsequence` or a greedy heuristic in the base case, depending on its size and complexity. Second, inverted alignments corresponding to gaps in the alignment are found, added to the syntenic path and annotated as non-translocated inversions. After that, alignments in remaining gaps are annotated as translocations, if available. Secondary inconsistent alignments of syntenic regions are likewise annotated as duplications. In a final step, cross-chromosomal translocations are identified and regions without any alignment (syntenic or otherwise) are annotated as highly diverged.
After finding genome structure, SyRI goes back to the raw alignment to identify small variants between the reference and sample. Both small and large variants are reported in a custom output format; small variants are also reported as a VCF file. SyRI's output can be visualized using the plotsr utility[124].

The Multi-Synteny Detector (msyd)

Using SyRI, structurally conserved and variable regions between a sample and reference assembly can be found in a straightforward and efficient manner. In my Bachelor's Thesis supervised by Manish Goel, I developed a tool to merge SyRI's synteny calls across multiple samples called against a common

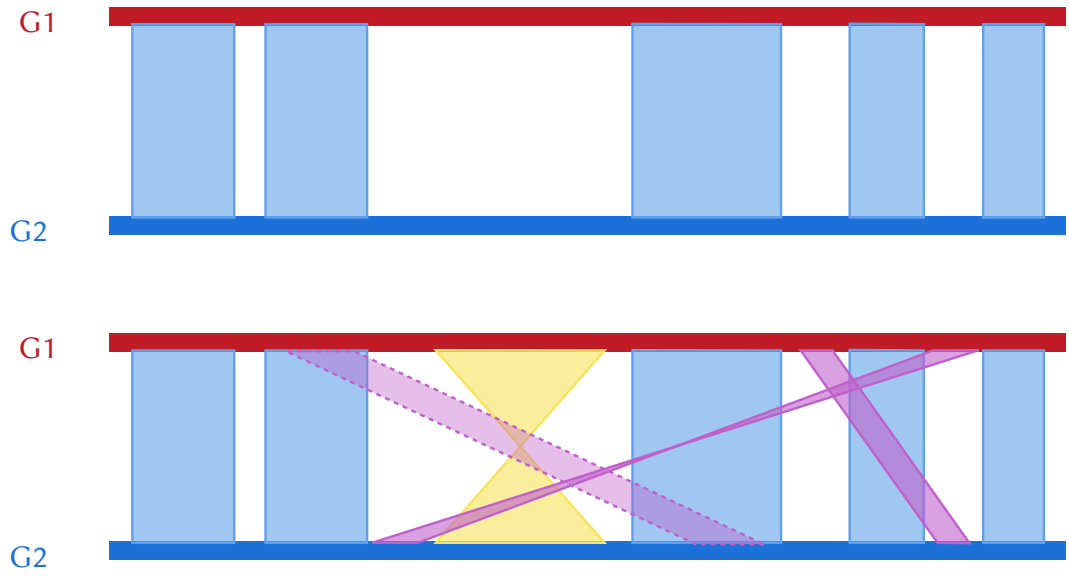


Figure 2: **Illustration of the SyRI algorithm.**
Top, in the first step, SyRI identifies a syntenic backbone of conserved genome structure between the reference and query.
Bottom, after identifying syntenic regions, leftover regions are known to be structurally variable, and are annotated appropriately in a multi-step process (see main text). Finding conserved structure first both makes the algorithmic problem tractable, and helps to avoid calling spurious similarities as structural variation. This is illustrated by the translocation drawn with a dashed outline (purple): here the large, syntenic alignments including both regions are a more parsimonious annotation than inserting a translocation based on the drawn secondary alignment.
Figure inspired by [123].

reference. Later, I worked on extending this tool with a realignment step to recover structural features not present on the reference, and named it the multisynteny detector (msyd). As msyd's structural similarity calls form the baseline for the synteny-centred graph representation of a pangenome described in this thesis, I will give an outline of msyd's core concepts. For more details on the msyd algorithm, the interested reader is referred to my Bachelor's Thesis and msyd's documentation, available in the GitHub repository.

The central concept of msyd is that of multisynteny. A multisyntenic region is defined as a set of at least two intervals on multiple, pairwise different samples such that any two intervals align to each other as part of SyRI's syntenic backbone when comparing the two samples. We further require multisyntenic regions to be maximal, defined as requiring that neither the multisyntenic region itself nor any set of subintervals can be extended to larger intervals or additional samples without violating the mutual synteny requirement. While a definition without requiring maximality would be consistent, it would allow multiple multisynteny annotations of the same dataset as well as overlapping multisyntenic regions. If a multisyntenic region encompasses all samples in a population/dataset, we term it to be core syntenic; otherwise we call it merasyntenic (from ancient Greek μερικως, meaning partial). Synteny is transitive; that is, if region a on sample A is syntenic to regions b, c on samples B, C, b also has to be syntenic to c, and vice versa, as synteny is always symmetric. This allows the efficient identification of all core syntenic regions: if we choose any sample as a reference to compare all other samples to, we can intersect the synteny calls on the common reference sample to obtain all core synteny with only $\mathcal{O}(n)$ required whole-genome alignments and SyRI runs. This will also identify any merasynteny that includes the chosen reference sample, but miss any for which this is not the case. In order to recover these missing annotations, msyd includes a realignment algorithm that iteratively

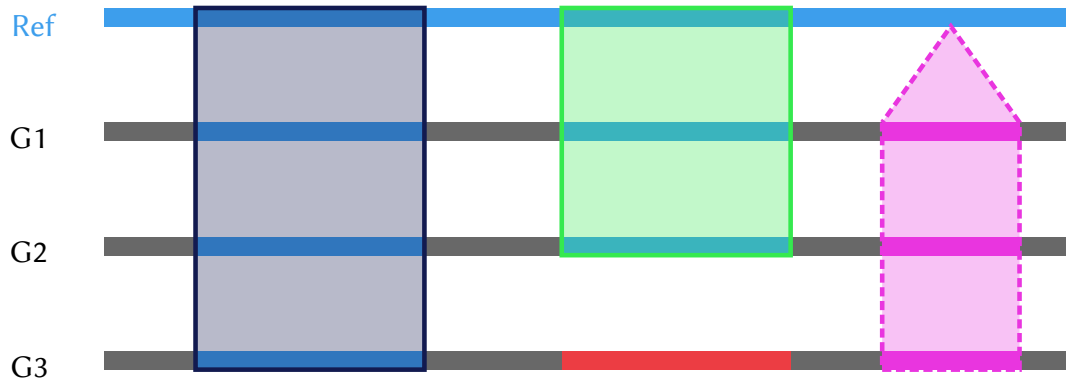


Figure 3: **Illustration of msyd's multisynteny concept.**
Left, *core syntenic* block or region. This has conserved structure in all samples, though it may still contain small variants. It can be identified very efficiently, regardless of the choice of reference.
Middle, *merasynteny* including the reference sample. This is identified as »bycatch« during the core synteny intersection step. It is conserved in some, but not all samples, and corresponds to an allele of a structural variant. In this case, the region indicated on G3 does not form part of a syntenic alignment to any other sample, and may be considered a *private* region.
Right, *non-reference merasyntenic* block. Structurally conserved in some samples not including the chosen reference, this region will be missed by the initial intersection step to the sample labelled as reference. In the illustrated case, it is deleted on the reference. It can be recovered by performing the intersection again on a different sample. Conceptually as well as biologically, there is no difference between merasynteny that includes and excludes the reference sequences, as it is merely the first arbitrarily chosen alignment target.

repeats the intersection procedure with different references until all multisynteny is found. This is performed separately for each gap between two coresyntenic regions – as they cover all samples, there cannot be a multisyntenic region that crosses a coresyntenic region –, avoiding expensive whole-genome alignments at this step, as it can require up to $\mathcal{O}(n^2)$ alignments.

To summarize the introduction, I have outlined the current state of the art in genome assembly, which has become routine in simple genomes, and more accessible than ever before in complex genomes. Phasing complex, polyploid genomes is still a significant challenge; in the first section of the Main, I will use modelling to show that read-based phasing is possible in principle also in polyploids, but additional information is still needed to completely phase complex genomes like the autotetraploid potato.
Once we have those large datasets of high-quality phased genomes, the question is how to make them usable, which requires a suitable representation of some kind. I will introduce a graph genome representation based on the concept of multisynteny as well as an efficient algorithm to compute it from a set of multisynteny calls. Given its strong conceptual foundation, this representation provides a number of guarantees such as acyclicity, having a linear backbone, a bounded number of edges per node and not allowing edges not local in at least one sample. Using these guarantees, a natural coordinate system can be defined for the representation that likewise maintains certain properties by respecting the global syntenic ordering between samples.
After concluding by describing a pipeline used to prepare input for the computation of the reference, I will discuss how this representation compares to the two leading sequence-based pangenome representations introduced above.

Main

As datasets of high-quality phased assemblies are required before computing any pangenome representation, I begin with the section deriving sequencing requirements for read-based genome phasing. This is inspired by our lab's recent assembly of 10 tetraploid, high quality potato genomes [42]. While the 10 cultivars were specifically chosen to be well-representative of the genetic material in European breeding programmes, obtaining more assemblies remains desirable, both to cover recent introgressions and to include South American landraces in the representation. These are well known in potato breeding as a source of resistance genes and for their great phenotypic diversity[125]. Single-platform phased assembly is now possible in humans, and would dramatically facilitate any efforts in this direction. To investigate whether this approach can also be applied to more complex genomes like potato, in the first section I derive minimal requirements for read-based phasing, and investigate the impact of different parameters upon these.

After deriving the minimal sequencing requirements, I will introduce the Multisyntenic Pangenome Graph (MPG) as a novel pangenome graph representation. The representation is centred on multisyntenic regions, which provides some guarantees that can be leveraged to prove that the representation has some desirable properties.
Using some of these properties, a straightforward and utile indexing scheme termed Pangenomic Coordinates (PanCo) can be defined in a very natural way. In the section, I also describe a simple and efficient algorithm for constructing a multisyntenic pangenome graph from a callset of multisyntenic regions, as well as for exporting it as a GFA file.

In the last section, I describe a pipeline implemented to perform the preprocessing required to obtain this representation efficiently in large datasets. Two datasets processed using his pipeline are briefly described. One is derived from the potato assemblies mentioned above, and the other uses the HPRC's phase 2 data release, to have a point of comparison to other pangenome representations[42, 83].

Phaseability

Given that a phased assembly of a human genome can nowadays be obtained with no additional inputs beyond ultra-long read sequencing[72, 82], the question naturally arises if this achievement can be extended to complex, polyploid genomes. Relying on only a single technology is highly desirable, as it simplifies experimental planning and reduces labour requirements, costs and possible failure modes, allowing higher sample throughput and shorter project timelines. This has, to the best of my knowledge, not yet been reported in the literature for any polyploid genome. The question naturally arises if this is because it is impossible with current sequencing technologies, and if so, what kind of advancements would be required to overcome this limitation. In this section, I will derive minimal requirements for long-read datasets to contain the information required to phase a genome. This naturally leaves out the question of whether current algorithms are able to successfully extract this information, but serves as a limit on the reliability of read-based phasing achievable by any conceiv-

able algorithm. If requirement minima turn out to be uneconomically high, it can be concluded that read-based phasing is not feasible in the modelled setting, and additional information from e.g. HiC experiments is required. The modelling effort may also serve to investigate how expected advances in sequencing technology affect our ability to use reads for phasing, in particular as reads get ever-longer.

In the literature, there are many empirically motivated suggestions for sequencing depth to use, some of them practically validated[12][80, 126, 48]. Sequencing depth recommendations are usually concerned either with the most economical way to genotype a cohort through resequencing, or a reliable way to obtain a high-quality assembly. With the recent possibility of phased assembly, some recommendations of sequencing inputs exist for humans, but none apply to polyploids nor separate phasing from assembly[80].
A literature search failed to find any recent theory-oriented work on sequencing coverage estimation. The most recent work in this direction seems to be Lander and Waterman's short paper on requirements for clone-based physical mapping in the second issue of Genomics in 1988 (ref. [127]). The paper introduced the use of the Poisson distribution to model random clones/sequencing reads, and its approach was later used to derive coverage requirements for the Human Genome Project[128].
In this section, I have a similar aim to it: To model the read-based phasing problem in a systematic way, and derive minimal dataset requirements for a genome to be phaseable with a certain probability. This depends on key parameters, in particular ploidy and the length of shared stretches, that can be systematically investigated using a theoretical model. To conclude, I will discuss the concrete situation in potato, reflecting our laboratory's current efforts to assemble potato genomes, and present a small script providing sequencing recommendations based on the approach developed in this section.

To approach this problem, some clarity on what phasing is (and is not) is required, in particular how it relates to genome assembly.
We term a position at which a haplotype is different from all others present in the host organism a *haplo-unique site*; they may correspond to single small variants, (parts of) larger variants, or clusters of multiple nearby variants that different haplotypes happen to have different alleles of. A phasing of a genome can be defined formally as a function taking as arguments two different haplo-unique sites, and returning whether they are found on the same molecule of DNA or not, thus sharing or not sharing a haplotype [13] . If we refer to the set of all haplo-unique sequences as U and a partition of U into haplotypes as $H := \{h_0, \ldots, h_n\}$, this can be expressed formally as:

$$\mathcal{P} : U \times U \to \{0, 1\}$$

$$\mathcal{P}(u_0, u_1) := \begin{cases} 1 & \text{if} \quad \exists h_i \in H : u_0, u_1 \in h_i \\ 0 & \text{otherwise} \end{cases}$$

In particular, $\mathcal{P}(\sqcap_\prime, \sqcap_\infty = 0$ for all u_0, u_1 not on the same chromosome. Defining phasing in terms of this boolean function rather than a haplotype partition directly has the advantage of being symmetric in regards to the arbitrary labeling of the haplotypes (i.e., $\mathcal{P}$ does not change if we decide to henceforth swap h_1 and h_2 or vice versa) and cleanly separating different chromosomes.
Using this definition of phasing, we can proceed to examine the information we have available in a dataset of sequencing reads, and proceed to find criteria by which we can decide if a reliable phasing can, in principle, be constructed from it. When encountering a read, we know all of its sequence to be derived from a single physical piece of DNA corresponding to a fragment of a single haplotype [14]. If

[12] Usually only in human.

[13] Note that in this definition, different chromosomes are always considered separate haplotypes; this reflects the fact that chromosomes segregate independently in offspring, but an alternative definition of a haplotype by inheritance from the same parent may permit haplotypes covering more than one chromosome

[14] With the possible exception of chimeric reads as a result of PCR artefacts or other sequencing errors.

two haplo-unique sequences u_0, u_1 are found on the same read $r := (u_0, u_1)$, we can assign them to the same haplotype; this construction can be transitively extended, if we have two reads $(u_0, u_1), (u_1, u_2)$, we can use the shared haplo-unique site u_1 to assign u_0 and u_2 to the same haplotype. Note that, repetitive sequences are not inherently problematic for phasing; so long as the sequence contains sufficient haplo-unique sequences, it can be easy to phase (though possibly difficult to assemble); and vice versa, a perfectly nonrepetitive sequence may be difficult or impossible to phase if it contains large stretches shared by all haplotypes.

We can thus derive a simple minimal requirement for reconstructing the phasing of a genome, given in theorem 1.

Theorem 1 *Given a set of reads $\mathcal{R}$ and a set of haplotypes $\mathcal{H}$, each modelled as consisting of an ordered set of haplo-unique sequences $\mathcal{H}_i := (u_0^i, u_1^i, \dots u_n^i)$, it is possible to construct a phasing $\mathcal{P}$ if and only if $\forall (u_j, u_{j+1}) \exists x \in \mathcal{R} : (u_j, u_{j+1}) \in x$, that is if every adjacent pair of haplo-unique sequences is covered by at least one read.*

A proof is simple to construct: if the condition is true, for any two haplo-unique sites on the same haplotype u_i, u_j with $i < j$, a chain of neighbouring haplo-unique sequences $u_i, u_{i+1}, \dots, u_{j-1}, u_j$ can be found and by the transitive closure $\mathcal{P}(u_i, u_j)$ must equal 1/true. If we know all neighbouring haplo-unique sequences to be covered, the absence of such a chain implies that u_i and u_j do not share a haplotype, and thus $\mathcal{P}(u_i, u_j)$ must evaluate to 0/false.

If the condition is violated for a pair of haplo-unique sequences u_i, u_{i+1} on the same haplotype, we cannot resolve them directly; transitive closure does not help either, as they are by construction directly adjacent.
If all but one haplotypes have been successfully phased out at a location, it is in principle possible to infer the phasing of all non-assigned haplo-unique sequences to the missing haplotype, recovering even haplo-unique sites not fulfilling the requirement given in Theorem 1. This can be seen most clearly in the case of a haploid genome present in only a single copy; in this case, clearly no information at all is required for assigning phase, as there is *a priori* only a single haplotype any sequence could be derived from.
However, considering practical limitations, having positive evidence for every haplotype may often be preferable; in this case, the condition in Theorem 1 is sharp. Otherwise, it may be relaxed to require a read only for all except one of the haplotypes.

Using some statistical modelling, we can turn this simple minimal requirement into an estimate of the minimal sequencing coverage required to confidently resolve the phasing of a genome. It should be noted that limitations in read accuracy and genome assembly and phasing algorithms may in practice impose somewhat higher requirements; however, calculating a lower bound can answer if single-technology phased assembly is at all possible, and provide guidance on which parameters are limiting factors and thus should prioritized for protocol optimization.

To ensure full phasing, the requirement of theorem 1 has to be met not just for every haplotype, but also for every position along the genome. In particular, this means that the mean sequencing coverage will require accounting for the following requirements (illustrated in Fig. 4):

1. every haplotype has to be sampled
2. every position along the genome has to be sampled, and
3. the read covering every position on every haplotype has to actually contain both adjacent haplo-unique sequences.

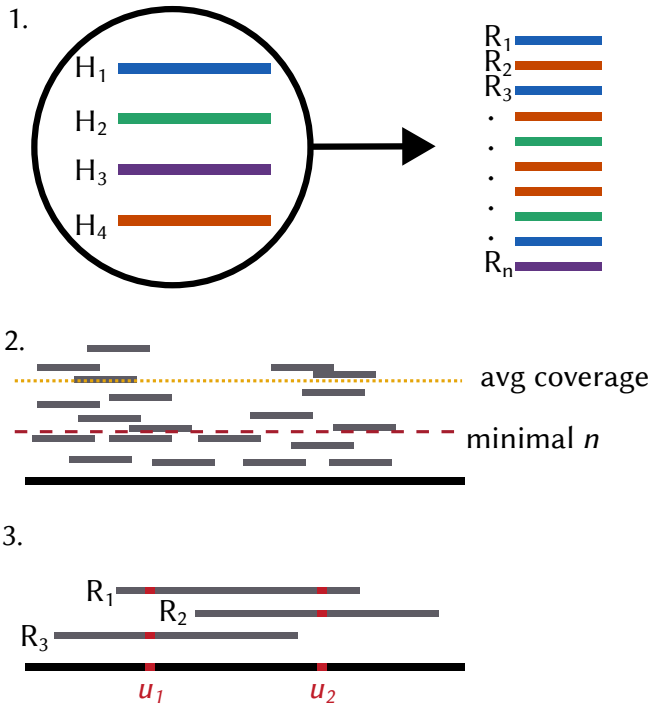


Figure 4: **Minimum coverage requirements for genome phasing.**
Step 1, every haplotype needs to be covered by a read. Because they are sampled randomly with replacement, more reads than haplotypes need to be drawn to be certain that all haplotypes are covered. **Step 2,** the average coverage has to be adjusted as to be sufficient to fulfil step 1's requirement even in regions that randomly have lower coverage than average.
Step 3, not every read is informative for phasing. In this illustration, R_2 and R_3 do not contain any information as they cover only a single haplotype. R_1, in contrast, can be used for phasing as it covers both u_1 and u_2.
For more information on the derivation and concrete results for each step, see the main text.

We can model each of these requirements in turn, allowing us to derive an overall lower bound.

To model the first requirement, if we consider a given position along the genome, every read mapping there will be drawn from all possible haplotypes with equal probability. Sampling is with replacement, if we assume our sequencing library to have sufficient size. Asking how many samples need to be drawn to cover every haplotype with confidence is equivalent to the Coupon Collector Problem, for which multiple approximations have been derived in the literature. For example, following the approach given below Lemma 3.8 in [129], we can estimate that given an upper bound α of the probability of not having sampled all n haplotypes, a sample of size at least $\lceil n(\log n - \log \alpha) \rceil$ is required.

The second requirement hearkens back to the early days of estimating sequencing requirements for whole-genome shotgun sequences – except instead of ensuring every position is covered at least once, we need every position to be covered in sufficient depth to make sure the first requirement is met. If we model observed coverage along the genome given a specified mean coverage using a Poisson distribution, we can derive a requirement for the mean coverage by solving for the quantile of the Poisson distribution. This corresponds exactly to the inverse of the upper Γ function, a numeric approximation of which is available in the `zipfR` package.

Finally, consider the effect of read length. First, it can trivially be seen that reads larger than the gap between adjacent haplo-unique sites are required in order to construct a phasing. But even reads of sufficient length may not be informative, if they happen to cover just one of the haplo-unique

Table 1: **Sequence coverage requirements for different parameters**.
In the first row is the ploidy of the organism. In the first column, the read length is given, expressed as multiples of the maximal length of sequences common across the haplotypes. If reads are shorter than this, no phasing can be produced. All values are computed with a PR bound of 20. Coverage values are rounded to the nearest whole number, and refer to haploid genome size.

$\frac{\ell}{d}$/**Ploidy**	2	3	4	6	10
1.5	45x	70x	95x	147x	252x
2.0	30x	47x	64x	98x	169x
4.0	20x	31x	42x	65x	112x
10.0	17x	26x	35x	54x	94x

sequences. We can compute the fraction of informative reads spanning both haplo-unique sequences for a given position from the fraction of valid starting positions, given that every read is uniquely specified by its starting position. A starting position is valid if a read of length ℓ starting from it spans both haplo-unique sites; all valid starting positions therefore have to be before the first haplo-unique sites, and there will be $\ell - d$ many, with d giving the distance between the two haplo-unique sequences and the length of each site assumed to be negligible compared to the read length. As every read has a single starting position, a mean sequencing coverage of X coverage corresponds to X/ℓ reads starting at every position along the genome in expectation. We thus expect $(\ell - d)\frac{X}{\ell} = \frac{\ell - d}{\ell}X$ reads to fulfil this criterion. As we assume reads to be Poisson distributed, we can simply multiply our coverage estimate derived in step 2 with this factor. Note that read length only matters here in how it relates to the distance between two haplo-unique sites. Therefore, d can be fixed as $d := 1$, and adjusted read length considered by giving ℓ in units of d, formally $\hat{\ell} := \frac{\ell}{d}$.

To make these estimates actionable, they were implemented in an R script, given in the Appendix. It takes a read length, maximal length of sequence in common to all haplotypes and a desired minimum probability for the phasing to contain the information required to resolve each shared segment. This probability is given as a Phred value and termed the (phasing) Reliability Score (RS). The script then calculates a minimum required coverage with reads of the given length, listing the impact of each of the steps outlined above. It also has support for estimating the number of required flowcells, given a genome size and ultralong yields per flowcell. Default values reflecting the potato genome are supplied for these.
Notably, the reliability score is interpreted as an upper bound, not as an average; that is, the sequencing requirements are calculated such that even the most challenging regions will meet the specified reliability. In easier regions with smaller haplotype-shared regions, the phasing may be siginificantly more reliable than the specified minimum.

Values for several representative cases are given in Table 1. Compare also Figure 5 for plots of different scenarios. In general, it may be observed that the desired phasing reliability and ploidy set a floor for the required coverage, while read lengh in relation to maximal distance between haplo-unique sites acts as a multiplier on this minimum. Desired reliability score contributes suprisingly little to this minimum, though its contribution increases as ploidy does. As read length approaches this distance, required sequencing depth increases rapidly. To avoid this unfavourable scaling, reads should be at least twice, ideally 3 times or more the maximal length of haplotype-shared sequences.

To provide some concrete minima for potato, a value for the size of regions common to different haplotypes is needed. Two studies have investigated this in the haplotypes of a single phased potato assembly each[130, 131]. They both report individual segments common to multiple haplotypes up to a megabase scale, but do not report exact length statistics or their frequencies; an analysis using

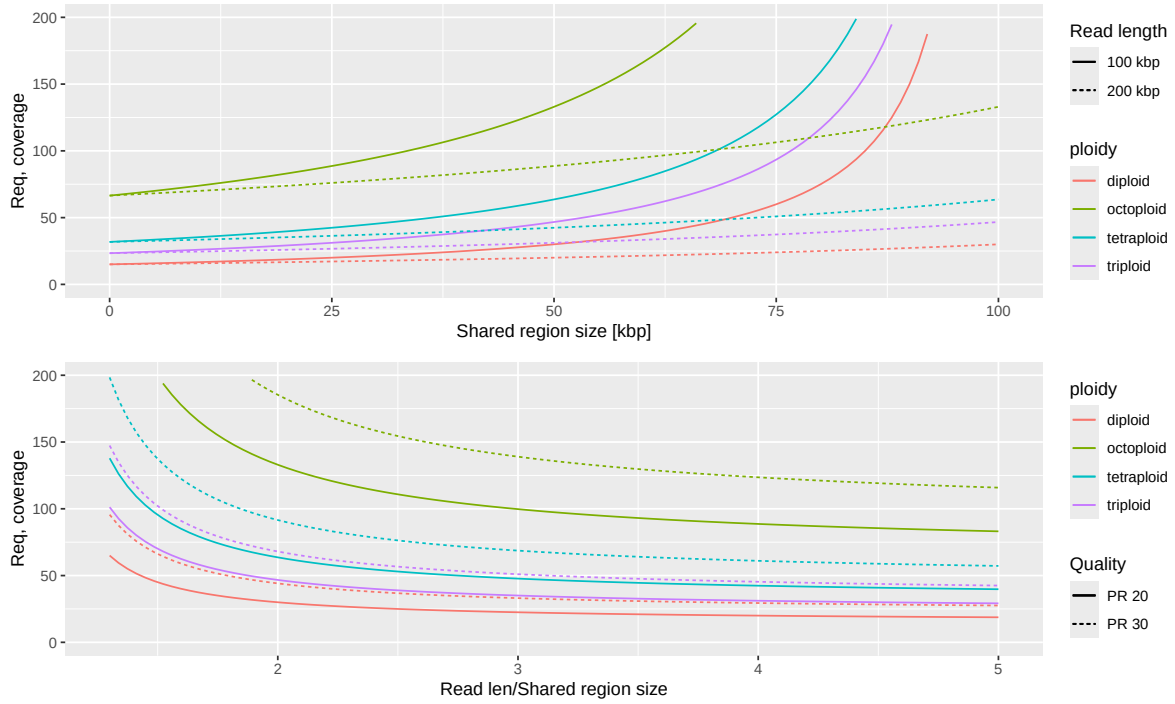


Figure 5: **Plot of required coverages in different scenarios.**
Top, minimal required coverage for multiple ploidies and read lengts against the maximal haplotype-shared region size. While ploidy matters in determining the base sequencing depth, the effect of read length outpaces it as the limit of shared regions surpassing read length is reached. All curves drawn using a PR of 20.
Bottom, minimal required coverage drawn against normalized read length (given in multiples of the shared region size) for different ploidies and desired reliability scores. The phasing at a reliability score of 30 requires more sequencing the more haplotypes are present in the sample, but even high phasing qualities of 30 are suprisingly accessible with sufficient read length.

the Otava genome reported in ref. [130] of 2619 shared segments larger than 10 kbp revealed roughly 75% of these to be below 100 kbp, and over 90 % to be smaller than 200 kbp (Craig Dent, personal communication). From Table 1, we can see that phasing through 100 kbp regions using 200 kbp reads while reaching a reliability score of 20 would require 64x coverage with ultralong reads. Relaxing the reliability score threshold to 10 (implying that 10% of breaks are expected to be unresolvable), 200 kbp regions could be phased with 54x coverage of 300 kbp reads.
To summarize, the full phasing of a potato genome would require phasing across megabase scale haplotype-shared regions and is not achievable with current sequencing and library preparation techniques. However, the majority of shared segments are much smaller, and can feasibly be resolved using high-coverage ONT ultralong sequencing.
Using the distribution of common segments described above, I estimate that an assembly using 54x coverage of 300 kbp reads would contain at least 200 breaks from large regions from regions larger than 200 kbp, 50 from unphased regions between 100 kbp and 200 kbp, and additional 20 from shorter regions. In total, this would imply an average of at least 6 phasing breaks per haplotype and chromosome. For some applications, particularly in plant breeding, this may already be sufficient; for applications requiring complete phasing, additional information could be used to fully phase the assemblies. While HiC data would be an obvious candidate, it might also be possible to use sample pedigrees or reference panels to resolve these breaks without additional sequencing.

The Multisyntenic Pangenome Graph

Using msyd, it is possible to comprehensively characterize multisynteny in a dataset of genome assemblies. Msyd reports its output in Population Synteny Format (PSF), a custom file format inspired by VCF [15]. A PSF record contains one mulitsyntenic region on one line, and starts with a BED-like position on the reference it has been aligned to, allowing tabix indexing. Following this, the position the multisyntenic region each sample and the alignment to the genomic region of the reference is given, with each sample separated by a TAB character and samples the multisyntenic region is not present in annotated as `.`, analogous to how small variants are reported in VCF format.
While this format is amenable to indexing and easy for users to parse in custom scripts, a significant downside to any custom file format is lacking integration with other tools. In the last two years, version 1 of the GFA format and its rGFA subset, originally developed to store intermediary representations of genome assembly, has increasingly become the standard for representing pangenomic datasets, while its . It is now supported by read-based genotypers, read and graph aligners as well as multiple visualization suites[118, 115]. Both `vg` and `gfatools` provide utilities for the file format[4, 3].
Given this developing software ecosystem and GFA's graph-based nature allowing greater flexibility than VCF format, finding a way to represent msyd's multisynteny calls in GFA1 format is desirable. In this section, I outline how multisynteny can be represented as a graph, how this was implemented efficiently in the multisynteny detector, and derive some properties of this representation that make it attractive compared to other graphical representations of pangenomic datasets. See the following section for how this representation can be used to define a pangenomic coordinate system, and the Discussion for its broader implications in the context of modern pangenomics and population genomics.

To see how to construct a graph representation of multisynteny, let us first consider how two multisyntenic regions A, B can relate to each other. In the following, we use $S(A)$ to refer to the set of samples A is present on, and A_i to refer to the interval representing A in sample i. A_i is undefined if $i \notin S(A)$.
First, no two regions can overlap each other on any organism unless $A = B$, as any overlap of two regions containing different samples would imply the overlapping region to be multisyntenic between all samples in $S(A) \cup S(B)$, violating the maximality condition. The same consideration makes it impossible for one multisyntenic region to contain another. A situation that could give rise to such an overlap would instead be annotated by msyd as separate, adjacent multisyntenic regions. Knowing that multisyntenic regions cannot overlap, we can define directed sample-adjacency. A and B are directedly i-adjacent, if there does not exist a multisyntenic region C such that $A_i < C_i < B_i$. To write i-adjacency, we use $A \prec_i B$. Directed sample-adjacency is an asymmetric relation, and any multisyntenic region can only ever be predecessor and successor to at most one other region per sample. We can further observe that it is impossible for A and B to »cross« each other, that is for the order of A and B's representative intervals to contradict each other on any two samples both A and B are present on, as this would violate the condition that both A and B be part of a syntenic whole-genome alignment. Formally, this requirement can be expressed as $A_i < B_i \implies \forall j \in S(A) \cap S(B) : A_j < B_j$. We can use this property to derive a sample-independent definition of directed adjacency: $A \prec B \iff \exists i \in S(A) \cap S(B) : \nexists C : A_i < C_i < B_i$. This relation is similar to sample-adjacency, but is undefined if A and B do not share a sample.

Using our definition of adjacency, we can now define a multisynteny graph. We first consider with

[15] Msyd also supports exporting to VCF format using symbolic allele annotations to store synteny calls alongside small variants, but symbolic alleles are not straightforward to parse and cannot represent all information present in msyd output, particularly alignments and non-reference multisyntenic regions.

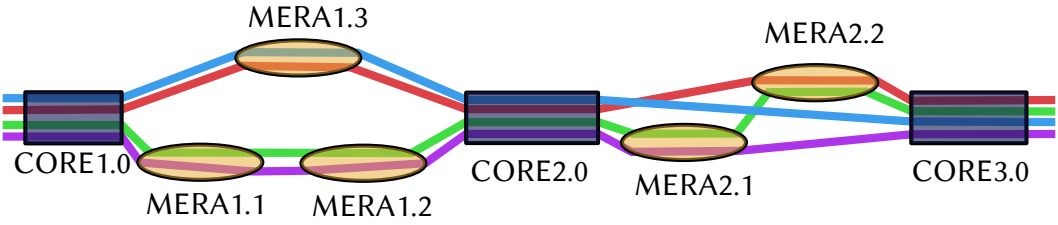


Figure 6: **Illustration of the Multisynteny Graph representation.**
This tube map shows an illustrative example of a multisynteny graph representation. Nodes are multisyntenic regions. Edges between nodes are colour-coded by the sample they occur on, allowing for multi-edges. Ingoing and outcoming edges corresponding to the same sample are connected, illustrating how the multisynteny graph consists of a merged set of paths. Note how coresyntenic regions induce bottlenecks in the graph, giving it an appearance like beads on a chain. If private nodes are added along edges representing sequence, a multisyntenic pangenome graph can be derived from this representation. Compare also Fig. 7 for a visualization of a real multisyntenic pangenome graph.
Nodes are indexed using PanCo indices, described later in the main text.

the set of all multisyntenic regions as the node set, and connect two nodes with a directed edge if they are directedly adjacent. The sequence assigned to the nodes corresponds to the sequence found at the sample used as reference during the identification of the multisyntenic regions; it could also be defined to be a consensus sequence derived from the alignments, though computing this would require an MSA, which may become prohibitively expensive as the number of samples grows larger. To include private regions in our representation, we can substitute an adjacency edge with a significant gap with a node containing the gap's sequence, and connect the target/source nodes to the new node instead. We call these private nodes, and other nodes multisynteny nodes, among which we can further differentiate core and merasyntenic nodes.
With this addition, the multisynteny graph becomes a pangenome graph; that is, for every sample there is an ordered set of nodes connected by directed edges such that the sequence obtained by walking along the path aligns well to the sample's genome, though small variants are not represented. In fact, this ordered set is guaranteed to be an acyclic path, as any node is either a multisyntenic node and thus part of a syntenic alignment to another genome or a private node with only a single incoming and outgoing edge.
Reflecting that the addition of private regions allows the graph to represent the sequence of the pangenome along with multisynteny calles, we term it *Multisyntenic Pangenome Graph (MPG)*.

Constructing a Multisynteny Graph

Internally, the graph is represented as a sorted list of nodes, with each node containing an adjacency list of its predecessors and successors, labelled by the sample on which they are directedly adjacent. This allows efficient forward and backward traversal in a simple datastructure. Private nodes are encoded implicitly in the genomic coordinates associated with each multisyntenic node. The graph is divided by chromosome, allowing all algorithms to be parallelized by chromosomes.
Since every node in the MPG has at most as many successors as there are samples, and the path of each sample through the graph is acyclic and mutually compatible, we can construct this graph representation using a simple linear scan through the multisyntenic regions. During the scan, we store a reference to the node indicating the current position on each sample. When encountering a new multisyntenic region, a new node is initialized containing predecessor-links to the current node on all samples the region contains. The successor-lists of these nodes are then updated to contain the newly initialized node, and it is added to the node list. This graph construction procedure avoids both lookahead and lookback by exploiting the nature of the MPG as union of a finite set of non-branching

```
1  Function Construct_MPG(multisyns)
2      curr := Map({s → null   ∀s ∈ get_samples() } )
3      for m ∈ multisyns.sort() do
4          n := init_node(m)
5          for s ∈ m.samples do
6              if dist(curr[s].multisyn[s], n.multisyn[s]) > T then
7                  // T is 0 by default, but can be configured
8                  make_link(curr[s], n)
9              else
10                 p := init_node(n.multisyn[s] - curr[s].multisyn[s])
11                 make_link(curr[s], p)
12                 make_link(p, n)
13                 yield p
14             end
15             curr[s] := n
16         end
17         yield n
18     end
19 end
```

Algorithm 1: Pseudocode illustrating the MPG construction algorithm.
T on line 5 is a parameter that can be adjusted by the user, if desired. The default setting is $T = 0$, meaning that even very small sequence between multisyntenic sequences are reported as private nodes. This maintains the invariant of not discarding any sequence during graph construction; if this is not a concern, setting it to e.g. 50 can result in a more compact graph representation.
The `make_link` function stores a bidirectional link in the predecessor/successor lists of both nodes.
The worst-case runtime of this algorithm is determined by the call to `.sort` in line 2; in practice the input received from msyd is for the most part presorted, and sorting can be ensured in linear time, making the average-case runtime $\mathcal{O}(|\text{get_samples}()||\text{multisyns}|$.

paths; to work successfully, it requires nodes to be scanned in an order compatible with the partial ordering derived from the directed adjacency relation. This can be ensured by simply sorting the multisynteny calls according to this relation prior to scanning.
A sketch of the algorithm for computing an MPG from a multisynteny callset is given in Algorithm 1.

The graph construction algorithm has expected and worst-case runtime complexity linear in the number of multisyntenic regions in the input and the number of samples. In the worst-case, ensuring sorting could take longer than the actual graph construction; the total worst-case runtime complexity is given by $\mathcal{O}(n|MS| + |MS| \log |MS|)$, with n as the number of samples and $|MS|$ as the number of multisyntenic regions in the input msyd callset. The first term is the graph construction step, which involves up to n steps for every multisyntenic region.
The second term comes from needing to ensure sorting of the multisyntenic regions prior to graph induction. As msyd's intersection algorithm ensures sorting, only switches to different realignment targets can introduce breaks in this sorting. This does not happen often in practice, so the appropriate output sorting is expected to take linear time in the average-case using an algorithm that can exploit prior sorted runs.
Memory-wise, the graph requires two references per sample and multisyntenic region beyond the base multisynteny representation, with only a constant single reference per sample additionally required during graph construction.

GFA export is performed by traversing the graph using the same order as during graph construction [16], though in the worst case as many private regions as samples need to be annotated in every traversal step. By default, the GFA format uses indexes introduced later in this section. See that section for an an overview of the GFA export procedure in Algorithm 2.
The GFA export supports using the W-line feature added in version 1.1 of the GFA standard to explicitly annotate all or a selection of sample's paths through the MPG. Further, the origin and samples present in a node are added to each node using the annotations defined in the rGFA extension of the GFA1 standard. Annotating sample paths as W-lines requires one graph traversal per sample, although only the nodes contained in the sample need to be visited at this stage.

The GFA export step has the highest memory requirements of all graph construction steps, despite favourable computational complexity. This derives from the need to access genomes from many samples to obtain the node sequences in the graph. Using `pysam`'s FASTA class for dynamically reading from the genome files as required helped reduce required memory, but demand can still be substantial in datasets with many samples and large genomes.

Properties of the multisynteny graph

After having defined and introduced the multisyntenic pangenome graph, we can now investigate some of its properties in more detail. First, we can see that the MPG is acyclic. Again consider two nodes in the graph A, B. Without loss of generality, we can assume both to be multisyntenic nodes, as any private node can only connect two multisyntenic nodes. Let a, b be the samples used as a reference when aligning A, B, and $S(X)$ and X_i describe the samples $X \in \{A, B\}$ is present on and its representative interval on sample i, respectively. This time, we will also be using the syntenic alignment $\mathcal{A}_{i\to j}$of two sample genomes i, j. We will treat the alignment as a function mapping an interval on sample i to the corresponding position on sample j. If the input interval does not have a syntenic alignment anywhere on sample j, the alignment shall map it to a zero-width interval immediately after the last interval that aligns. Note that $\mathcal{A}$ is a monotonic function though not necessarily injective or surjective. The definition of multisynteny requires that $\mathcal{A}_{i\to j}(A_i) = A_j$ for all samples $i, j \in S(A)$ for a multisyntenic region A.
We assume A and B to induce a cycle, that is for there to be both a path $A \to C \to \cdots \to B$ and $B \to \cdots \to C' \to A$. Private nodes along this path can be ignored, as they are essentially extended edges between multisyntenic nodes. By the construction of the multisynteny graph, these paths imply that $A \prec C \prec \cdots \prec B \prec \cdots \prec C' \prec A$. For every one of these steps, there must be at least one sample ℓ_i such that $X^i_{\ell_i} < X^{i+1}_{\ell_{i+1}}$, with X^i as the multisyntenic region at position i along the path. Since alignments are monotonic, and intervals of multisyntenic regions have to align to each other for all samples the region is present in, we can write each these steps in terms of an alignment to any sample. We further know that both $\ell_0, \ell_i \in S(A)$. We write

$$\begin{aligned} A_i < X^0_i < \mathcal{A}_{\ell_i\to i}(X^1) < \cdots < \mathcal{A}_{\ell_i\to i}(X^i) < \mathcal{A}_{\ell_i\to i}(A_{\ell_i}) \\ A_i < \cdots < \mathcal{A}_{\ell_i\to i}(A_{\ell_i}) \\ A_i < A_i \end{aligned}$$

, in the last step using that all regions in A have to align to each other. This is obviously a contradiction, proving that the graph is acyclic.

This fact allows us to further characterize the MPG. As every path has to pass through coresyntenic nodes exactly once and the graph is acyclic, the coresyntenic nodes induce regular bottlenecks in the

[16] At no additional computational cost, as the sorting is retained in the node list.

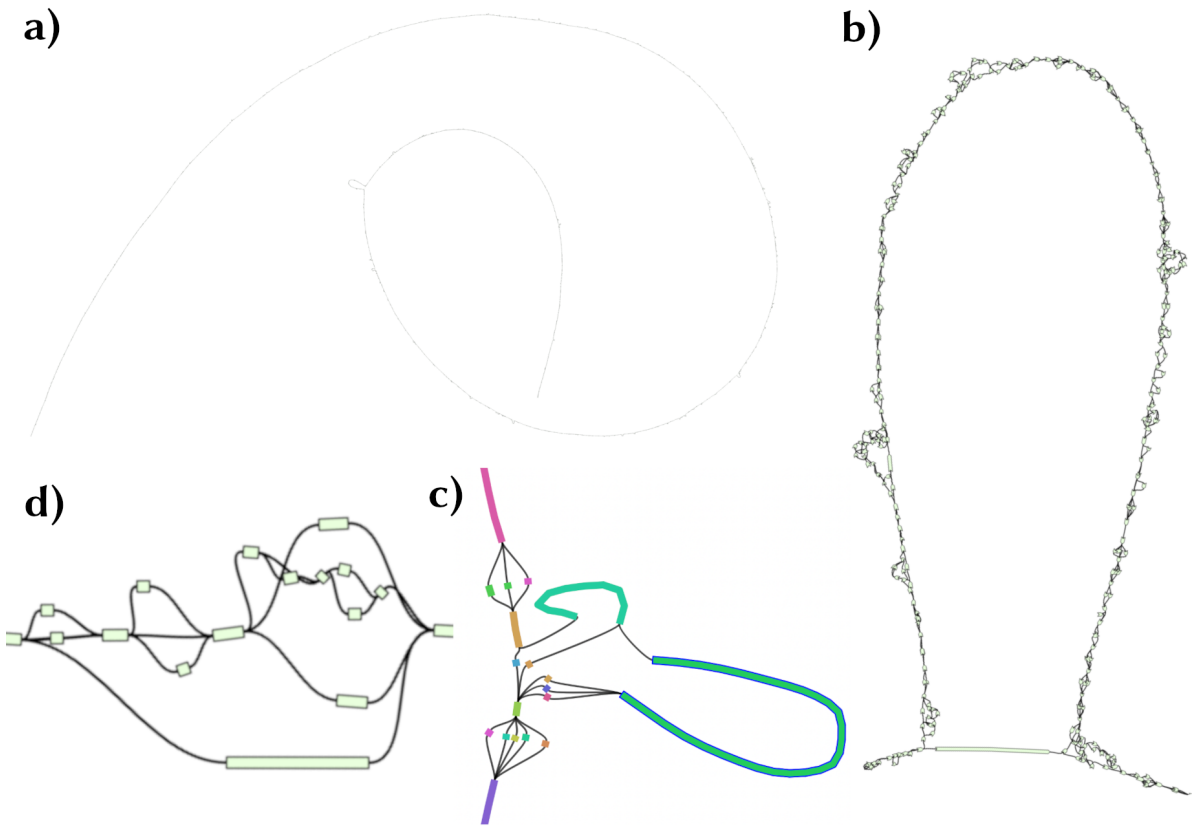


Figure 7: **Bandage visualization of the Multisyntenic Pangenome graph of a testing dataset.** This MPG was computed in a testing dataset composed of 5 genomes from the AMPRIL population[132, 133]. It was computed with realignment disabled, so is likely undercollapsed. **a),** the MPG of chromosome 1. Note the linear structure and lack of nonlocal connections in the graph. **b),** a zoomed-in of a structurally variable region of chromosome 1. Node length corresponds to the length of the associated sequence. The large node near the bottom is a 321 kbp private region in the c24 sample. **c, d),** smaller examples of structural variation in the graph. Note the private nodes with similar length sharing both a predecessor and a successor as possibly undercollapsed (as this graph was generated from non-realigned data). **a, b, d)** were visualized using GFAviz[117]. **c)** was visualized using BandageNG[115, 116]. In **c)** and **d)**, the layout was manually adjusted for enhanced clarity.

graph, giving it a linear backbone. Subgraphs between two coresyntenic regions form (super-)bubbles, as the only ingress/egress nodes are the coresyntenic regions itself. This guarantees that there are no non-local edges in the MPG.
Nodes always have equal in- and outdegree, except for those at contig breaks or telomere ends in T2T assemblies. The degree of a node is also bounded by the number of samples passing through it; as such, no node can have more adjacent edges than twice the number of samples. As the start/end of each node corresponds to the start/end of a syntenic alignment, all non-private nodes with less than two successors represent deletions.
This bound on the number of edges is a useful property, as many classic graph algorithms scale linearly in the number of edges.

In the next section, I will outline a procedure using the regular structure of the MPG to define a pangenomic coordinate system termed PanCo, allowing genomic coordinates to be compared across different samples in a principled and consistent way. This not only serves a direct purpose, but also demonstrates the advantages of a representation possessing strong structural guarantees, and illustrates how these can be used to derive efficient algorithms appropriate for modern genomics datasets.

```
Function GFA_Export(nodes, samples)
    index := PanCo()
    // already sorted from construction
    for n ∈ nodes do
        if n.multisyn is coresyn then
            index := index.increment_core()
        else
            index := index.increment_mera()
        end
        n.index := index
        emit_S_line(n.index, get_sequence(n.multisyn)) for p ∈ n.predecessors do
            // p must have been indexed, as it is a predecessor
            emit_L_line(p.index, n.index)
        end
    end
    for s ∈ samples do
        path := trace_path(s, nodes)
        emit_W_line(s, path)
    end
end
```

Algorithm 2: Pseudocode illustrating the GFA export of a MPG, using PanCo indexes as node IDs.

Note that we get the PanCo indexes »for free« from the ordering of the graph during construction. In the implementation, they are computed during graph construction; this was left out in Algorithm 1 for clarity. Here they are used as node IDs to make the GFA output easier to read. The most expensive step in this algorithm in practice is obtaining the genomic sequence at the required position, as this involves significant I/O.

Pangenomic Coordinates (PanCo)

To navigate a genome representation for any use, a system allowing for positions to be specified and compared is required. In a linear reference genome, a chromosome ID and number corresponding to the number of bases from the 3' end is sufficient for this purpose. This is a powerful and intuitive index: it supports both qualitative and numeric comparisons – intuitively, the expressions $a < b$ and $|a - b|$ when comparing two positions a, b –, and can seamlessly be extended to describe intervals as well as individual positions.
In designing a navigation aid for a graph-based representation of multiple genomes, it would be preferable to keep as many of these features as possible while remaining simple to understand and efficient to implement. In this subsection, I will introduce Pangenomic Coordinates (PanCos), an addressing system exploiting the Multisyntenic Pangenome Graph's structural guarantees to provide an interpretable, straightforward way to specify and compare positions in all or some of the genomes simultaneously.

A PanCo consists of two natural numbers, and uniquely identifies a node in the MPG when combined with a chromosome ID. Positions within the node can be given analogous to a linear reference, as a natural number indicating the distance from the 3' end on the sample used as reference for the MPG node. Using the alignments that induced the multisynteny, this can be mapped to syntenic positions in all samples present in the multisyntenic region. PanCos are typically written in the form `27.3` or `17.0` when working within a chromosome, or as `Chr04:27.3` otherwise. Alternatively, #

as a separator, to provide compatibility with the HPRC's Pangenome Sequence Naming convention [17][134]. For easier human readability, a PanCo is prefixed by msyd with the letters `MERA`, `CORE` or `PRIV` dependending on its status when writing output files.

The first number of the PanCo indicates the preceding coresyntenic node; if there is none, it is set to 0. This can only happen to multisynteny found between the start of a chromosome and the first multisyntenic regions, often representing structural diversity in subtelomeric repeat arrays. As coresyntenic nodes have a total ordering and induce bottlenecks in the graph, the first number can be used to compare PanCos. That is, given two PanCos `A.a`, `B.b` and $\texttt{A} > \texttt{B}$, $\texttt{A.a} > \texttt{B.b}$ follows regardless of the values of `a` and `b`. The second number indexes merasyntenic nodes found between two core syntenic nodes. As the MPG is acyclic, its directed edges imply a partial ordering. The second number's are assigned to be consistent with this partial ordering; that is, $\texttt{A} = \texttt{B}$ and $\texttt{a} > \texttt{b}$ implies $\texttt{A.a} \not< \texttt{B.b}$, meaning that either $\texttt{A.a} > \texttt{B.b}$ on all shared samples, or the two do not share any samples and cannot be compared by transitive closure of intermediary nodes, either. As the preceding coresyntenic node covers all samples occurs before any merasyn, , its second number is assigned as 0; it follows that $\texttt{A.0} < \texttt{A.x}$ for $x > 0$. In general, coresyntenic PanCos always end in 0, and merasyntenic or private PanCos always have a non-zero number at the second position (`0.0` is undefined to avoid ambiguity).

Computing PanCos for coresyntenic regions is trivial, and can be done in linear time. For merasyntenic regions, an asymptotically efficient way to obtain a valid ordering would be to compute a topological ordering of the subgraph between the preceding and following coresyntenic node, e.g. by depth-based traversal. This has linear runtime in the number of nodes and edges in the subgraph; as the number of edges per node is bounded by the number of samples, the overall runtime is $\mathcal{O}(n|V|)$, with n as the number of samples and $|V|$ as the number of merasyntenic nodes in the subgraph. However, in practice a consistent ordering can be obtained much faster and without even using the graph structure, simply by sorting multisyntenic regions using a simple comparator: $A < B$ iff $\exists i \in S(A) \cap S(B) : A_i < B_i$, with $A < B$ undefined if A and B do not share a sample. It was shown earlier in this section that this ordering has to be consistent across all samples; it is in fact identical to the transitive closure of the directed adjacency defined as the MPG was introduced, and is already computed along with the graph representation. All that is left to do, then, is to scan through all nodes and increment two counters for core and merasynteny, resetting the latter counter when incrementing the first.

PanCo has partial support for describing ranges: those corresponding to coresynteny boundaries can easily be described by their two PanCo indexes $\texttt{S.0} < \texttt{E.0}$, with any node with PanCo `A.a` being part of the range iff $\texttt{S.0} < \texttt{A.a} < \texttt{E.0}$ holds. Ranges inside of merasyntenic nodes are harder to work with, in particular because indexes may not be consecutive. If we define a range between two nodes as the intersection of their set of (indirect) predecessors and successors, respectively, the inclusion criterion above will not always be sharp, i.e. while all nodes within the range will meet the criterion, other nodes may do so as well. As a practical approach, I would advise using node-sets or paths instead of ranges when referring to subgraphs of the MPG not matching coresyntenic bubbles. While arguably more cumbersome, their more explicit nature provides for easier debugging and avoids ambiguity.

[17] Some gfa tools require nodes to have numeric IDs; in this case, `000` may be used as a separator that will maintain unique IDs in typical cases.

Dataset Preparation

To build a multisyntenic pangenome graph, a multisynteny callset is required. To compute this callset, msyd requires a synteny callset of every sample against a common reference.. This can be computed with SyRI, and in turn requires a whole-genome alignment to the reference. Msyd also requires these alignment files in order to retrieve the alignments of the multisyntenic regions. For both realignment and graph construction, the genomic sequence of all samples is also required. Msyd takes these inputs in samplesheet format, with one line per sample containing all files for that sample.

In order to generate the inputs required for msyd at scale in large datasets, a nextflow-based pipeline was implemented to perform alignments and SyRI runs, along with a script to automatically generate msyd's samplesheet from the pipeline output (see Appendix 20). These preprocessing steps are typically much more computationally demanding than the msyd run itself; implementing them as a nextflow pipeline allows efficient parallelization across multiple nodes of a HPC environment[18].

Given the aim to make the pipeline scale to today's large pangenomic datasets, several improvements were made to the implementation of msyd's core functionality, reflecting both usability and performance improvements, including improved multithreading. In particular, the core datastructures were rewritten to avoid SyRI-style monolithic pandas DataFrames and instead use a Cython list type per chromosome. This new datastructure has a more convenient API, but more importantly can be efficiently serialized and thus allows parallelization across different chromosomes without the interference of Python's Global Interpreter Lock. In total, this allows msyd to use multiple cores, especially for steps other than the sequence alignment.

To maintain it in a usable state for the pipeline as well as general use, SyRI was updated to be compatible with the 3.x version of `pandas`, and run in versions of python newer than 3.11. This required a partial rewrite of SyRI's internal datastructures and the way Cython's TypedMemoryView interface is used to account for pandas' change to copy-on-write behaviour in version 3. Along the way, some of SyRI's error handling was modernized, particularly to avoid silent errors and always return a non-zero exit code in any error state.
All changes made to SyRI were upstreamed, and will be incorporated into the 1.8 release.

For software development and debugging, a previously prepared dataset based on the high-quality genome assemblies of the AMPRIL population described in refs. [132, 133] was used. Two datasets were prepared using the pipeline described above, one from human and one from potato genomes.

The human dataset was prepared from the HPRC's Phase 2 data release of >200 chromosome-scale phased assemblies, downloaded in Novembre 2025[83]. This presents both a large dataset to test the graph's performance, and allows comparison to other pangenome graph representations, which were developed and primarily tested on human assemblies. As some assemblies had chromosome breaks, they were first scaffolded to the CHM13 T2T reference using RagTag[135]. After that, the pipeline described above was run for all samples, using minimap2's `asm10` preset.

The potato dataset was prepared from the European potato pangenome reported in ref. [42]. It contains 10 haplotype-resolved assemblies for different cultivars for a total of 40 distinct haplotypes. Maximally representative cultivars have been selected for assembly based on pedigrees from historical breeding records and short-read genotyping. Altogether, the dataset is estimated to cover 85% of the haplotype space present in modern European potatoes not contributed by recent introgression breeding. The potato dataset both reflects the lab's research interest, and provides a stress-test of the multisyntenic pangenome graph: potato genomes are very challenging to work with due to their

[18] And provides robustness to jobs being cancelled when other jobs take more than their allocated memory, a persistent problem on the MPIPZ cluster

high repeat content, extensive structural and small-scale diversity, and a breeding history shaped by ancient and modern introgressions.

When testing the pipeline, msyd's realignment step fell out as being often unreliable, frequently not reporting any new synteny during a realignment step. Manual inspection revealed several cases where clear, segregating haplotypes were missed, particularly in short gaps. Further testing indicated that minimap2 did not report any alignment in the analysed regions, likely as a result of not finding enough minimizers. While a MPG can still be constructed from a non-realigned dataset, it will represent non-reference sequences as private even though they may be shared.

Discussion

Phaseability

By modelling read-based phasing systematically, estimates for minimal required coverage to phase genomes could be obtained in different scenarios. The impact of multiple parameters was assessed, and revealed read length compared to the distance between haplo-unique sites to be the limiting factor, with ploidy providing a floor for the minimal required coverage as read length approaches infinity.
In order to be economically feasible, read length should be at least twice, ideally three times or more of the maximal expected distance between haplo-unique sites.

This demonstrates that polyploidy does not pose a fundamental hurdle to read-based phasing; in fact, breeding history, particularly recent introgressions and inbreeding, may actually play a bigger role in determining phaseability than ploidy, as it can cause large regions shared by to be shared by multiple haplotypes. This is the case in potato, where some shared regions up to several megabasepairs in size have been reported. However, even in potato most shared regions are much smaller, (Craig Dent, unpublished). These are accessible to read-based phasing, meaning that a partial phasing containing < 10 phasing breaks per haplotype and chromosome is likely achievable using current sequencing technology. Given that a read-based phasing can be obtained at no additional cost given that ultralong reads are required for genome assembly in any case, it will be interesting to investigate if a partial phasing using them can relax the requirements for other approaches; in particular, it could allow complete phasing at far lower HiC coverage and with less manual intervention than required so far[131].

The minima provided by the modelling-based approach presented in this thesis are concordant with recent empirical recommendations in human, considering that the read-length correction can be neglected in humans [19]. While the theoretical minimum is calculated to be 15x without read length correction at a reliability score of 20, ref. [80] recommends 15-20x coverage of ONT-Ultralong (along with 10x of HiC data).

As in any modelling approach, some simplifications and assumptions had to be made in order to provide mathematical tractability and interpretability.
First, it was assumed that haplo-unique sites can be perfectly identified in any read spanning them. How reasonable this assumption is depends on the nature and length of the haplo-unique site and the error profile of the sequencing technology. Assuming that unique k-mers are used as haplo-unique sites, the error rate of modern ONT sequencing should be sufficiently low as to not make this a major concern for realistic values of k. For example, an ONT read with a quality score of 30 has a $> 90\%$ chance of containing a 1000-mer with no errors. In this sense, the read error rate

[19] At an average distance of 1 kbp between SNPs and no complex breeding history, this is a reasonable assumption when considering >100 kbp Ultralong reads

constrains the maximal length of probes used to detect haplo-unique sites. If this becomes a relevant constraint, probes designed to accomodate sequencing errors such as gapped k-mers or strobemers may be used[136, 137].
Modelling the impact of imperfect identification of haplo-unique sites would be fairly straightforward; similar to the read length correction described in this thesis, an additional step multiplying by the inverse of the likelihood of misidentifying any of the two required haplo-unique sites could be added. Second, the modelling assumed all reads to have equal length. This is clearly not the case in real data. Given the sensibility of the coverage requirements to read length, accounting for varying read length may provide significantly sharper estimates, especially in species with large common regions at the limit of phaseability. As a pragmatic recommendation, the coverage calculator can be run with multiple read lengths and results compared to find an ideal way to reach the desired phasing quality, as demonstrated in the paragraph of the Main text assessing the phaseability of the potato genome in the »Phaseability« section. As a more concrete example, if a single ultralong sequencing run gives 5x coverage of the target genome with >100 kbp reads, of which 2x are $\geq$200 kbp reads, the number of flowcells that the coverage calculator estimates for each of these two read lengths can be computed, and the lower number chosen for sequencing.
From a statistics point of view, this could be modelled by a more sophisticated read length correction, by computing a per-length correction factor and applying it to the read length distribution, then reporting the expected value required to reach sufficient coverage. In practice, users may not always have a suitable read length distribution at hand, as it depends on library preparation and may vary between different sequencing runs.

Finally, the modelling done in this thesis provides a minimum requirement for the dataset to contain sufficient information to phase a genome. To actually turn this into a phasing, the phasing signal needs to be extracted by an algorithm. In diploids, current algorithms seem to be capable of reaching this minimum, as evidenced by the success of read-based phased assembly in human[80, 82].
Recent developments in ONT-Ultralong protocols suitable for plant samples provide an opportunity to test if current tooling can also reach this limit in complex, polyploid genomes[49].
If this is indeed the case, it may facilitate phased plant genome assembly by either eliminating or reducing the need for additional information such as HiC sequencing. If it is not, it provides an opportunity for tool developers to design novel algorithms. It should be noted that none of the leading phased assembly tools were designed with phased assembly or ultralong reads in mind; as such, there may yet be significant potential of these datasets that we are not currently able to utilize. Tool development efforts should also strive to integrate phasing information from reads with that of orthogonal information, particularly increasingly accessible long-read chromosome conformation methods like PoreC and CiFi[82, 138, 139].

Multisyntenic Pangenome Graphs

Comparing to pangenome graphs based on local alignments and variation graphs, the Multisyntenic Pangenome Graph representation differs in a number of key ways. First, MPGs are acyclic by construction, which other leading approaches cannot guarantee. This dramatically simplifies the graph structure, and allows the use of more efficient algorithms for many common operation. MPGs can by construction never have nonlocal connections, as all edges represent adjacency on at least one sample. Incidentally, this implies that different chromosomes will always form disjoined subgraphs in the MPG, allowing graph operations to be parallelized across different subgraphs/chromosomes. This differentiates the MPG approach from other pangenome graphs that may introduce both cycles and nonlocal connections, particularly in repetitive regions or when encountering inversions.

An MPG conserves the genome ordering of all samples; as remarked in the proof of acyclicity, the $<$

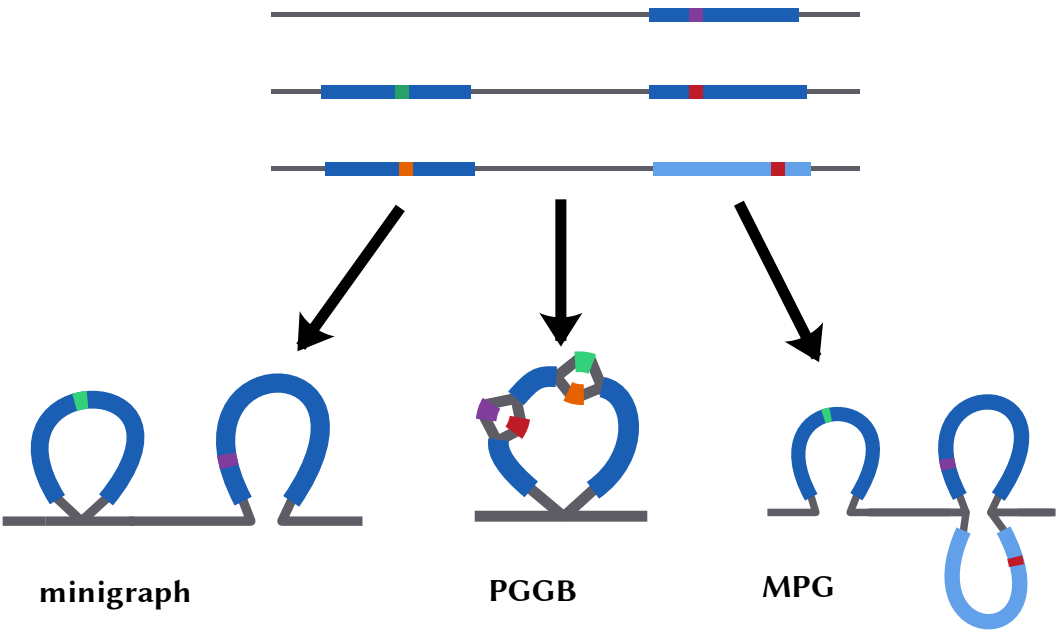


Figure 8: **Illustration of graphs produced by minigraph and PGGB, and the MPG.**
Top, the input genomes used to construct the representations. Blue, thick ribbons are identical segments; the light blue ribbon is inverted compared to the other segments. On these ribbons, small variants are drawn in a different colour.
Left, the minigraph representation. Small variants are not represented in minigraph pangenomes, though they can be added using the minigraph-cactus pipeline. Minigraph collapses inversions in the same place into a single node, but does not collapse duplicated segments.
Middle, the PGGB pangenome graph. PGGB collapses identical sequences, even if they are not adjacent. Small variants are represented as nodes in the graph, and are merged across identical segments.
Right, the MPG representation of the haplotypes in the top. Similar to minigraph, the MPG does not represent small variants. It also does not collapse inversions, given that they present breaks in syteny. Note that it is the only graph representation that is acyclic.
Figure adapted from Extended Data Fig. 3 of ref. [39].

comparator on any sample cannot be violated by the graph's child-of relation, as $<$ is the transitive closure of the per-sample directed adjacency $\prec_i$. As it conserves the sample-wise ordering of all pairwise or multisyntenic regions, an MPG can be viewed as a datastructure storing a partial ordering. When comparing two nodes, if one of them is a child of the other, that implies that the parent is ordered first. If neither is the other's child, their comparison is undefined in the partial ordering. Taking this view, the Pangenome Coordinates (PanCo) »fall out« of the graph representation very naturally as any arrangement of all multisyntenic regions that is consistent with this partial ordering implied by the MPG. As the MPG is a directed acyclic graph, this corresponds to a topological ordering, which can be found efficiently.

PanCo not only reflects a practical way to navigate the MPG and efficiently compare coordinates with another, but also highlights the advantages of a pangenome representation with strong theoretical guarantees as the basis of simple, efficient algorithms.
Compare also the ease at which a MPG can be indexed with PanCos to the difficulty of indexing nodes in a variation graph: the odgi sort command implements multiple cycle-breaking algorithms in order to then traverse the entire graph and assign a topological ordering, alongside a force-directed-layout based stochastic embedding approach[118]. This is necessary for variation graphs, because their construction by local alignments does not provide the same guarantees as the MPG approach does. I am not aware of any indexing designed for minigraph's or minigraph-cactus' graph genome.

In general, the MPG representation has stricter criteria for collapsing parts of the raw haplotypes than other pangenome graphs. In order to be collapsed, regions have to not just align to another but form part of a syntenic whole-genome alignment in multiple samples. Requiring whole-genome synteny is what allows the MPG to have a linear backbone without nonlocal connections or cycles, however it may also lead the MPG to undercollapse regions when they are mistakenly not identified as syntenic. While the MPG construction itself uses an exact algorithm, its inputs in the form of alignments and synteny calls use heuristic procedures that may miss real synteny. Minimap2's approximate alignment may fail in highly repetitive or significantly diverged regions, while SyRI's greedy heuristic may not always find the largest chain of alignments to call as a syntenic backbone.
This underlies the anecdotal observation that some regions of the MPG seem like they could safely be collapsed further without violating any of its guarantees based on manual inspection. This is desirable, as it would lead to a more compact and easier-to-visualize representation, while keeping all of the MPG's strengths. Strategies for achieving this are discussed in the next section.

Future Steps

Another clear next step is to investigate and visualize the multisyntenic pangenome graph representation of the potato dataset. Behaviour of the representations about regions known to be structurally variable, such as introgressed regions and regions containing NLR genes would be a particular point of interest. It would further be interesting to see whether it is possible to use the minigraph-cactus pipeline to »spikify« the MPG graph, recovering small variants[100]. As the file format is compatible and MPG graphs do not represent small variants similar to minigraph's output, in principle this should be possible, and may be a way to leverage the strengths of different approaches facilitated by a common software ecosystem.
Another approach would be to use the fact that multisyntenic regions have exact sequence positions on all of their samples as well as alignments of these regions to their representative sample to construct a consensus sequence for each node. A Centre*-based approximative MSA could be computed efficiently given that the sequences are already aligned to a common reference. A consensus could be derived either by column-wise majority vote, or represented as a sequence profile using HMM-based alignment algorithms[140].
While the cactus-based approach would be faster to develop and benefit from cactus' efficient MSA approach[141], the availability of a per-node MSA may be interesting for population genetic studies, and profile-based alignments have proven very efficient and sensitive in both DNA and protein alignment[122, 142].

To reduce the degree of undercollapsing observed in the MPG, an improvement to the reliability of msyd's realignment functionality would likely be a major step. The main limitation during the realignment step seems to be the alignment itself: minimap2 often reports no alignments, even when manual inspection reveals sequences across different haplotypes to be almost base-level identical. minimap2's minimizer approach works less well when aligning smaller sequences; additionally, spacers inserted during the extraction of sequences for realignment may disrupt minimap2's alignment[143]. As no synteny can be called without any alignments, the regions are falsely reported as private.
While some degree of undercollapsing may be unavoidable while maintaining computational efficiency, it is still generally desirable to collapse regions showing multisynteny. Two strategies are possible to reduce the occurrence of undercollapsing. Either to tune the pipeline of the graph construction's inputs, or to introduce an additional step after graph induction that further collapses the graph, somewhat similar to smoothxg's role in the PGGB pipeline[111].
Porting the realignment step to use a Dynamic Programming-based aligner, for example Parasail, may make it more reliable[144]. For short sequences, this should not significantly change runtime;

for longer sequences, perhaps a fallback to minimap2 or another heuristic approach may still be desirable. An alternative would be a naïve, length-based merging of haplotypes, similar to that used by Jasmine[145]. While this would be a very computationally efficient approach, it would likely fail to resolve nested variation, a key goal of modern pangenomic representations.
A postprocessing solution would require more conceptual innovation, especially to ensure that all collapses introduced by the postprocessing step reflect real, global synteny and do not violate the MPG guarantees. A starting point could be selecting nodes sharing a predecessor and successor, and collapsing them if their sequences align. Forcing collapsed nodes to share an adjacent node would ensure that no cycles are introduced during the collapse operation – to introduce a cycle by merging two nodes in an acyclic graph, one of the nodes must be a child of the other, which cannot be the case if they share both a predecessor and a successor.

Given the increasing ease of assembling the genome of larger samples, it becomes desirable to have approaches capable of performing comparisons at a representation level. One approach could be to use the Pangenomic Coordinate system as a linearization of the MPG that maintains consistency with all input haplotypes, providing a consensus genome structure. Comparing consensus linearized genomes across populations or species could enable insight into how genome structure evolves in separated populations. Structural variation is commonly thought to be a major factor in speciation after early work by Dobzhansky and Müller found cryptic polymorphisms to mediate reproductive incompatibility in Drosophila[146, 147].
Recently, some structural variants have been found to be maintained polymorphic across an entire clade[148]. This highlights that a principled way to account for within-species polymorphism is required also in cross-species comparisons, and provides an opportunity to study natural experiments of homologous alleles evolving in isolation.
A similar approach could also provide a novel framework to perform intra-species association studies, if the populations are chosen as subgroups of a sample showing or not showing a certain trait. To estimate the heritability of the trait, the consensus linearizations of the splits could be compared using e.g. whole-genome-alignment algorithms or k-mer based approaches, and compared to random splits to account for population structure. If this comparison can be made per locus, it could also map associations to certain regions of the MPG.

In closing, the dramatic improvements in sequencing and assembly technology the last few years have seen continue to pay dividends in the form of larger and higher-quality assembly datasets. With efforts like the Earth Biogenome Project, genomic resources are increasingly also available in species not traditionally used as a model organism.
Representing, analysing and ultimately making sense of these heterogeneous, large datasets presents many challenges, demanding innovative approaches, efficient and scalable implementations and good questions to ask of these datasets. All of these aspects can benefit from a strong conceptual grounding; in this thesis, I hope to have contributed to two aspects of this, by modelling when a phased genome assembly is possible from a read dataset and proposing a novel, synteny-oriented approach to graph genome representations.

Acknowledgements

It has been exciting to see empirical and theoretical approaches crosspollinate each other in a mixed dry and wet lab, both during this thesis and before in my Bachelor's Thesis. The environment has been tremendously stimulating, and I feel that I have grown a lot from being exposed to it and having many great conversations. I would like to thank all current and past Schneebergerlab members for creating such a friendly and cooperative atmosphere.

In particular, I thank Craig Dent for his supervision, consistently bonzer feedback and willingness to bounce around crazy ideas. I also thank Korbinian Schneeberger for the impulse to pivot away from k-mers early during this project, and for fostering a friendly and supportive lab environment.
For discussions specifically regarding this project, I would like to thank Xiao Dong, Manish Goel and Matthew Parker.
Outside the lab, I am grateful for Aleix Canalda-Baltrons, Matthew Hahn and Jens Zentgraf for insightful discussions and feedback about.
Lastly, I would like to thank the Cologne Snack Desk for tirelessly ensuring appropriate blood glucose and durian levels.

Bibliography

[1] Guo, D. *et al.* A pangenome reference of wild and cultivated rice. *Nature* **642**, 662–671 (2025). URL https://doi.org/10.1038/s41586-025-08883-6.

[2] Avni, R. *et al.* A pangenome and pantranscriptome of hexaploid oat. *Nature* **649**, 131–139 (2026). URL https://doi.org/10.1038/s41586-025-09676-7.

[3] Li, H. Gfatools github (2019). URL https://github.com/lh3/gfatools.

[4] Garrison, E. *et al.* Variation graph toolkit improves read mapping by representing genetic variation in the reference. *Nature Biotechnology* **36**, 875–879 (2018). URL https://doi.org/10.1038/nbt.4227.

[5] Mendel, G. *Versuche über Pflanzen-Hybriden* (Brünn, Im Verlage des Vereines, 1866, 1866). URL https://www.biodiversitylibrary.org/item/124139. Https://www.biodiversitylibrary.org/bibliography/61004 — Caption title. — Written in pencil on recto of front free endpaper: RC Punnett. — Dibner Library. Heralds of science (1980 ed.), | 35.

[6] Portin, P. & Wilkins, A. The evolving definition of the term "gene". *Genetics* **205**, 1353–1364 (2017). URL https://doi.org/10.1534/genetics.116.196956. https://academic.oup.com/genetics/article-pdf/205/4/1353/49469224/genetics1353.pdf.

[7] Sturtevant, A. H. The linear arrangement of six sex-linked factors in drosophila, as shown by their mode of association. *Journal of Experimental Zoology* **14**, 43–59 (1913). URL https://onlinelibrary.wiley.com/doi/abs/10.1002/jez.1400140104. https://onlinelibrary.wiley.com/doi/pdf/10.1002/jez.1400140104.

[8] Dobzhansky, T. Genetical and cytological proof of translocations involving the third and the fourth chromosomes of drosophila melanogaster. *Biol. Zbl* **49**, 408–419 (1929).

[9] McClintock, B. The origin and behavior of mutable loci in maize. *Proceedings of the National Academy of Sciences* **36**, 344–355 (1950). URL https://www.pnas.org/doi/abs/10.1073/pnas.36.6.344. https://www.pnas.org/doi/pdf/10.1073/pnas.36.6.344.

[10] Avery, O. T., MacLeod, C. M. & McCarty, M. Studies on the chemical nature of the substance inducing transformation of pneumococcal types. Inductions of transformation by a desoxyribonucleic acid fraction isolated from pneumococcus type III. *Journal of Experimental Medicine* **149**, 297–326 (1979). URL https://doi.org/10.1084/jem.149.2.297. https://rupress.org/jem/article-pdf/149/2/297/1267712/297.pdf.

[11] Watson, J. D. & Crick, F. H. C. Molecular structure of nucleic acids: A structure for deoxyribose nucleic acid. *Nature* **171**, 737–738 (1953). URL https://doi.org/10.1038/171737a0.

[12] Nirenberg, M. W. & Matthaei, J. H. The dependence of cell-free protein synthesis in e. coli upon naturally occurring or synthetic polyribonucleotides. *Proceedings of the National Academy of Sciences* **47**, 1588–1602 (1961). URL https://www.pnas.org/doi/abs/10.1073/pnas.47.10.1588. https://www.pnas.org/doi/pdf/10.1073/pnas.47.10.1588.

[13] Roeder, R. G. 50+ years of eukaryotic transcription: an expanding universe of factors and mechanisms. *Nature Structural & Molecular Biology* **26**, 783–791 (2019). URL https://doi.org/10.1038/s41594-019-0287-x.

[14] Lipman, D. J. & Pearson, W. R. Rapid and sensitive protein similarity searches. *Science* **227**, 1435–1441 (1985). URL https://www.science.org/doi/abs/10.1126/science.2983426. https://www.science.org/doi/pdf/10.1126/science.2983426.

[15] Pearson, W. R. & Lipman, D. J. Improved tools for biological sequence comparison. *Proceedings of the National Academy of Sciences* **85**, 2444–2448 (1988). URL https://www.pnas.org/doi/abs/10.1073/pnas.85.8.2444. https://www.pnas.org/doi/pdf/10.1073/pnas.85.8.2444.

[16] Initiative, T. A. G. Analysis of the genome sequence of the flowering plant arabidopsis thaliana. *Nature* **408**, 796–815 (2000). URL https://doi.org/10.1038/35048692.

[17] Chinwalla, A. T. *et al.* Initial sequencing and comparative analysis of the mouse genome. *Nature* **420**, 520–562 (2002). URL https://doi.org/10.1038/nature01262.

[18] Lander, E. S. *et al.* Initial sequencing and analysis of the human genome. *Nature* **409**, 860–921 (2001). URL https://doi.org/10.1038/35057062.

[19] Venter, J. C. *et al.* The sequence of the human genome. *Science* **291**, 1304–1351 (2001). URL https://www.science.org/doi/abs/10.1126/science.1058040. https://www.science.org/doi/pdf/10.1126/science.1058040.

[20] Perbal, L. The case of the gene. *The EMBO Reports* **16**, 777–781 (2015). URL https://doi.org/10.15252/embr.201540179.

[21] Collins, F. S., Green, E. D., Guttmacher, A. E., Guyer, M. S. & on behalf of the US National Human Genome Research Institute. A vision for the future of genomics research. *Nature* **422**, 835–847 (2003). URL https://doi.org/10.1038/nature01626.

[22] Altshuler, D., Donnelly, P. & Consortium, T. I. H. A haplotype map of the human genome. *Nature* **437**, 1299–1320 (2005). URL https://doi.org/10.1038/nature04226.

[23] Li, H., Ruan, J. & Durbin, R. Mapping short DNA sequencing reads and calling variants using mapping quality scores. *Genome Res* **18**, 1851–1858 (2008).

[24] Kent, W. J. BLAT–the BLAST-like alignment tool. *Genome Res* **12**, 656–664 (2002).

[25] McKenna, A. *et al.* The genome analysis toolkit: a MapReduce framework for analyzing next-generation DNA sequencing data. *Genome Res* **20**, 1297–1303 (2010).

[26] Weigel, D. & Mott, R. The 1001 genomes project for arabidopsis thaliana. *Genome Biology* **10**, 107 (2009). URL https://doi.org/10.1186/gb-2009-10-5-107.

[27] Durbin, R. M. *et al.* A map of human genome variation from population-scale sequencing. *Nature* **467**, 1061–1073 (2010). URL https://doi.org/10.1038/nature09534.

[28] Shabalin, A. A. Matrix eqtl: ultra fast eqtl analysis via large matrix operations. *Bioinformatics* **28**, 1353–1358 (2012). URL https://doi.org/10.1093/bioinformatics/bts163. https://academic.oup.com/bioinformatics/article-pdf/28/10/1353/48866250/bioinformatics_28_10_1353.pdf.

[29] GA4GH. Hts specs (2025). URL https://samtools.github.io/hts-specs/.

[30] Wang, Y., Zhao, Y., Bollas, A., Wang, Y. & Au, K. F. Nanopore sequencing technology, bioinformatics and applications. *Nature Biotechnology* **39**, 1348–1365 (2021). URL https://doi.org/10.1038/s41587-021-01108-x.

[31] Mahmoud, M., Agustinho, D. P. & Sedlazeck, F. J. A hitchhiker's guide to long-read genomic analysis. *Genome Res* **35**, 545–558 (2025).

[32] Logsdon, G. A., Vollger, M. R. & Eichler, E. E. Long-read human genome sequencing and its applications. *Nature Reviews Genetics* **21**, 597–614 (2020). URL https://doi.org/10.1038/s41576-020-0236-x.

[33] Hu, J. *et al.* Nextpolish2: A repeat-aware polishing tool for genomes assembled using hifi long reads. *Genomics, Proteomics & Bioinformatics* **22**, qzad009 (2024). URL https://doi.org/10.1093/gpbjnl/qzad009. https://academic.oup.com/gpb/article-pdf/22/1/qzad009/59350449/qzad009.pdf.

[34] Mastoras, M. *et al.* Highly accurate assembly polishing with DeepPolisher. *bioRxiv* (2024).

[35] Lewin, H. A. *et al.* Earth biogenome project: Sequencing life for the future of life. *Proceedings of the National Academy of Sciences* **115**, 4325–4333 (2018). URL https://www.pnas.org/doi/abs/10.1073/pnas.1720115115. https://www.pnas.org/doi/pdf/10.1073/pnas.1720115115.

[36] Rhie, A. *et al.* Towards complete and error-free genome assemblies of all vertebrate species. *Nature* **592**, 737–746 (2021). URL https://doi.org/10.1038/s41586-021-03451-0.

[37] Nurk, S. *et al.* The complete sequence of a human genome. *Science* **376**, 44–53 (2022). URL https://www.science.org/doi/abs/10.1126/science.abj6987. https://www.science.org/doi/pdf/10.1126/science.abj6987.

[38] Rhie, A. *et al.* The complete sequence of a human y chromosome. *Nature* **621**, 344–354 (2023). URL https://doi.org/10.1038/s41586-023-06457-y.

[39] Liao, W.-W. *et al.* A draft human pangenome reference. *Nature* **617**, 312–324 (2023). URL https://doi.org/10.1038/s41586-023-05896-x.

[40] Paten, B., Novak, A. M., Eizenga, J. M. & Garrison, E. Genome graphs and the evolution of genome inference. *Genome Res.* **27**, 665–676 (2017).

[41] Human Pangenome Project. Hprc working groups (2026). URL https://humanpangenome.org/working-groups/.

[42] Sun, H. *et al.* The phased pan-genome of tetraploid european potato. *Nature* **642**, 389–397 (2025). URL https://doi.org/10.1038/s41586-025-08843-0.

[43] Yang, L. *et al.* Phased-assembly-driven pangenome graphs for structural variant genotyping and complex trait mapping in dairy cattle. *Nature Communications* **17**, 2186 (2026). URL https://doi.org/10.1038/s41467-026-68807-4.

[44] Coronado-Zamora, M., Salces-Ortiz, J. & González, J. Drosomics: a comparative genomics browser to explore omics data in natural populations of d. melanogaster. *bioRxiv* (2022). URL https://www.biorxiv.org/content/early/2022/11/24/2022.07.22.501088. https://www.biorxiv.org/content/early/2022/11/24/2022.07.22.501088.full.pdf.

[45] Fang, B. & Edwards, S. V. Pangenomes: new tools for ecological and evolutionary genomics. *Trends in Ecology & Evolution* **41**, 230–244 (2026). URL https://www.sciencedirect.com/science/article/pii/S0169534725003441.

[46] Anderson, S. Shotgun dna sequencing using cloned dnase i-generated fragments. *Nucleic Acids Research* **9**, 3015–3027 (1981). URL https://doi.org/10.1093/nar/9.13.3015. https://academic.oup.com/nar/article-pdf/9/13/3015/6166406/9-13-3015.pdf.

[47] Green, P. Against a whole-genome shotgun. *Genome research* **7**, 410—417 (1997). URL http://genome.cshlp.org/content/7/5/410.full.pdf.

[48] Earth Biogenome Project. Earth biogenome sequencing recommendations (2026). URL https://www.earthbiogenome.org/report-on-assembly-recommendations.

[49] Lu, D. *et al.* Nanopore ultra-long sequencing and adaptive sampling spur plant complete telomere-to-telomere genome assembly. *Molecular Plant* **17**, 1773–1786 (2024). URL https://doi.org/10.1016/j.molp.2024.10.008.

[50] Pacific Biosciences. How does hifi sequencing work? (2026). URL https://www.pacb.com/technology/hifi-sequencing/how-it-works/.

[51] Oxford Nanopore Technologies. How nanopore sequencing works (2026). URL https://nanoporetech.com/platform/technology/basecalling.

[52] Pacific Biosciences Inc. Highly accurate single-molecule long reads (2018). URL https://www.pacb.com/press_releases/pacific-biosciences-announces-a-new-paradigm-in-dna-sequencing-highly-a

[53] Pacific Biosciences Inc. Pacbio announces major advances for revio and vega to lower genome cost and expand multiomic capabilities (2025). URL https://www.pacb.com/press_releases/pacbio-announces-major-advances-for-revio-and-vega-to-lower-genome-cost-and-expand-mul

[54] Pacific Biosciences Inc. Pacbio to begin commercial shipment of revio systems (2023). URL https://www.pacb.com/press_releases/pacbio-to-begin-commercial-shipment-of-revio-systems/.

[55] Oxford Nanopore Technologies. Dorado release notes (2022). URL https://github.com/nanoporetech/dorado/releases/tag/v0.0.1.

[56] Oxford Nanopore Technologies. R10.3: the newest nanopore for high accuracy nanopore sequencing (2020). URL https://nanoporetech.com/news/news-r103-newest-nanopore-high-accuracy-nanopore-sequencing-now-available-store.

[57] Sanderson, N. D. *et al.* Comparison of r9.4.1/kit10 and r10/kit12 oxford nanopore flowcells and chemistries in bacterial genome reconstruction. *Microbial Genomics* **9** (2023). URL https://www.microbiologyresearch.org/content/journal/mgen/10.1099/mgen.0.000910.

[58] Stanojević, D., Lin, D., Florez de Sessions, P. & Šikić, M. Telomere-to-telomere phased genome assembly using error-corrected simplex nanopore reads. *bioRxiv* (2024). URL https://www.biorxiv.org/content/early/2024/05/21/2024.05.18.594796. https://www.biorxiv.org/content/early/2024/05/21/2024.05.18.594796.full.pdf.

[59] Jain, M. *et al.* Nanopore sequencing and assembly of a human genome with ultra-long reads. *Nature Biotechnology* **36**, 338–345 (2018). URL https://doi.org/10.1038/nbt.4060.

[60] Oxford Nanopore Technologies. From kilobases to "whales": a short history of ultra-long reads and high-throughput genome sequencing (2021). URL https://nanoporetech.com/blog/news-blog-kilobases-whales-short-history-ultra-long-reads-and-high-throughput-genome.

[61] Fleischmann, R. D. *et al.* Whole-genome random sequencing and assembly of *Haemophilus influenzae* rd. *Science* **269**, 496–512 (1995). URL https://www.science.org/doi/abs/10.1126/science.7542800. https://www.science.org/doi/pdf/10.1126/science.7542800.

[62] Pop, M., Salzberg, S. & Shumway, M. Genome sequence assembly: algorithms and issues. *Computer* **35**, 47–54 (2002).

[63] Compeau, P. E. C., Pevzner, P. A. & Tesler, G. How to apply de bruijn graphs to genome assembly. *Nature Biotechnology* **29**, 987–991 (2011). URL https://doi.org/10.1038/nbt.2023.

[64] Huang, X. A contig assembly program based on sensitive detection of fragment overlaps. *Genomics* **14**, 18–25 (1992).

[65] Huang, X. & Madan, A. CAP3: A DNA sequence assembly program. *Genome Res* **9**, 868–877 (1999).

[66] Green, P. Phrap documentation (1996). URL http://bozeman.mbt.washington.edu/phrap.docs/phrap.html.

[67] Wong, J. *et al.* Linear time complexity de novo long read genome assembly with goldrush. *Nature Communications* **14**, 2906 (2023). URL https://doi.org/10.1038/s41467-023-38716-x.

[68] Pevzner, P. A., Tang, H. & Waterman, M. S. An eulerian path approach to dna fragment assembly. *Proceedings of the National Academy of Sciences* **98**, 9748–9753 (2001). URL https://www.pnas.org/doi/abs/10.1073/pnas.171285098. https://www.pnas.org/doi/pdf/10.1073/pnas.171285098.

[69] Kececioglu, J. D. & Myers, E. W. Combinatorial algorithms for dna sequence assembly. *Algorithmica* **13**, 7–51 (1995). URL https://doi.org/10.1007/BF01188580.

[70] Rautiainen, M. *et al.* Telomere-to-telomere assembly of diploid chromosomes with verkko. *Nature Biotechnology* **41**, 1474–1482 (2023). URL https://doi.org/10.1038/s41587-023-01662-6.

[71] Cheng, H., Concepcion, G. T., Feng, X., Zhang, H. & Li, H. Haplotype-resolved de novo assembly using phased assembly graphs with hifiasm. *Nature Methods* **18**, 170–175 (2021). URL https://doi.org/10.1038/s41592-020-01056-5.

[72] Cheng, H., Asri, M., Lucas, J., Koren, S. & Li, H. Scalable telomere-to-telomere assembly for diploid and polyploid genomes with double graph. *Nature Methods* **21**, 967–970 (2024). URL https://doi.org/10.1038/s41592-024-02269-8.

[73] Antipov, D. *et al.* Verkko2 integrates proximity-ligation data with long-read de bruijn graphs for efficient telomere-to-telomere genome assembly, phasing, and scaffolding. *Genome Res* **35**, 1583–1594 (2025).

[74] Renkawitz, J., Lademann, C. A. & Jentsch, S. Mechanisms and principles of homology search during recombination. *Nature Reviews Molecular Cell Biology* **15**, 369–383 (2014). URL https://doi.org/10.1038/nrm3805.

[75] Peñalba, J. V. & Wolf, J. B. W. From molecules to populations: appreciating and estimating recombination rate variation. *Nature Reviews Genetics* **21**, 476–492 (2020). URL https://doi.org/10.1038/s41576-020-0240-1.

[76] Healey, A. L. *et al.* The complex polyploid genome architecture of sugarcane. *Nature* **628**, 804–810 (2024). URL https://doi.org/10.1038/s41586-024-07231-4.

[77] Mason, A. S. & Wendel, J. F. Homoeologous exchanges, segmental allopolyploidy, and polyploid genome evolution. *Frontiers in Genetics* **Volume 11 - 2020** (2020). URL https://www.frontiersin.org/journals/genetics/articles/10.3389/fgene.2020.01014.

[78] The International Wheat Genome Sequencing Consortium (IWGSC) *et al.* Shifting the limits in wheat research and breeding using a fully annotated reference genome. *Science* **361**, eaar7191 (2018). URL https://www.science.org/doi/abs/10.1126/science.aar7191. https://www.science.org/doi/pdf/10.1126/science.aar7191.

[79] Campoy, J. A. *et al.* Gamete binning: chromosome-level and haplotype-resolved genome assembly enabled by high-throughput single-cell sequencing of gamete genomes. *Genome Biology* **21**, 306 (2020). URL https://doi.org/10.1186/s13059-020-02235-5.

[80] Sarashetti, P., Lipovac, J., Tomas, F., Šikić, M. & Liu, J. Evaluating data requirements for high-quality haplotype-resolved genomes for creating robust pangenome references. *Genome Biology* **25**, 312 (2024). URL https://doi.org/10.1186/s13059-024-03452-y.

[81] Cheng, H. *et al.* Efficient near-telomere-to-telomere assembly of nanopore simplex reads. *Nature* (2026). URL https://doi.org/10.1038/s41586-026-10105-6.

[82] Gross, C. *et al.* Single-platform nanopore sequencing enables diploid telomere-to-telomere genome assembly and haplotype-resolved 3d chromatin maps. *bioRxiv* (2026). URL https://www.biorxiv.org/content/early/2026/03/21/2026.03.19.712851. https://www.biorxiv.org/content/early/2026/03/21/2026.03.19.712851.full.pdf.

[83] Human Pangenome Project. Hprc data release 2 (2025). URL https://humanpangenome.org/hprc-data-release-2/.

[84] Human Pangenome Project. Hprc timeline (2026). URL https://humanpangenome.org/release-timeline/.

[85] Consortium, T. . G. P. *et al.* The 1001g+ project: A curated collection of arabidopsis thaliana long-read genome assemblies to advance plant research. *bioRxiv* (2024). URL https://www.biorxiv.org/content/early/2024/12/29/2024.12.23.629943. https://www.biorxiv.org/content/early/2024/12/29/2024.12.23.629943.full.pdf.

[86] Nassir, N. *et al.* A draft uae-based arab pangenome reference. *Nature Communications* **16**, 6747 (2025). URL https://doi.org/10.1038/s41467-025-61645-w.

[87] Wang, Y. *et al.* The 1000 chinese pangenome empowers medical and population genetics. *Nature* (2026). URL https://doi.org/10.1038/s41586-026-10315-y.

[88] Lian, Q. *et al.* A pan-genome of 69 arabidopsis thaliana accessions reveals a conserved genome structure throughout the global species range. *Nature Genetics* **56**, 982–991 (2024). URL https://doi.org/10.1038/s41588-024-01715-9.

[89] Igolkina, A. A. *et al.* A comparison of 27 arabidopsis thaliana genomes and the path toward an unbiased characterization of genetic polymorphism. *Nature Genetics* **57**, 2289–2301 (2025). URL https://doi.org/10.1038/s41588-025-02293-0.

[90] Matthews, C. A., Watson-Haigh, N. S., Burton, R. A. & Sheppard, A. E. A gentle introduction to pangenomics. *Briefings in Bioinformatics* **25**, bbae588 (2024). URL https://doi.org/10.1093/bib/bbae588. https://academic.oup.com/bib/article-pdf/25/6/bbae588/60702740/bbae588.pdf.

[91] Rojas-Miranda, H., Madrigal-Ly, V. & Molina-Mora, J. A. Benchmarking genome assemblers for four bacterial models based on contiguity, correctness, and completeness. *Scientific Reports* **15**, 42858 (2025). URL https://doi.org/10.1038/s41598-025-26847-8.

[92] Sigaux, F. [cancer genome or the development of molecular portraits of tumors]. *Bulletin de l'Academie nationale de medecine* **184**, 1441—7; discussion 1448—9 (2000). URL http://europepmc.org/abstract/MED/11261250.

[93] Bottery, M. J. Ecological dynamics of plasmid transfer and persistence in microbial communities. *Current Opinion in Microbiology* **68**, 102152 (2022). URL https://www.sciencedirect.com/science/article/pii/S1369527422000364.

[94] Rodríguez-Beltrán, J., DelaFuente, J., León-Sampedro, R., MacLean, R. C. & San Millán, Á. Beyond horizontal gene transfer: the role of plasmids in bacterial evolution. *Nature Reviews Microbiology* **19**, 347–359 (2021). URL https://doi.org/10.1038/s41579-020-00497-1.

[95] Lovell, J. T. *et al.* Genespace tracks regions of interest and gene copy number variation across multiple genomes. *eLife* **11**, e78526 (2022). URL https://doi.org/10.7554/eLife.78526.

[96] Noll, N., Molari, M., Shaw, L. P. & Neher, R. A. Pangraph: scalable bacterial pan-genome graph construction. *Microbial Genomics* **9** (2023). URL https://www.microbiologyresearch.org/content/journal/mgen/10.1099/mgen.0.001034.

[97] Canalda-Baltrons, A. *et al.* Genome graphs reveal the importance of structural variation in mycobacterium tuberculosis evolution and drug resistance. *Nature Communications* **16**, 10746 (2025). URL https://doi.org/10.1038/s41467-025-65779-9.

[98] Ebler, J. *et al.* Pangenome-based genome inference allows efficient and accurate genotyping across a wide spectrum of variant classes. *Nature Genetics* **54**, 518–525 (2022). URL https://doi.org/10.1038/s41588-022-01043-w.

[99] NCBI. Blast databases documentation (2020). URL https://ftp.ncbi.nlm.nih.gov/blast/documents/blastdb.html.

[100] Hickey, G. *et al.* Pangenome graph construction from genome alignments with minigraph-cactus. *Nature Biotechnology* **42**, 663–673 (2024). URL https://doi.org/10.1038/s41587-023-01793-w.

[101] Sierra, P. & Durbin, R. Identification of transposable element families from pangenome polymorphisms. *Mobile DNA* **15**, 13 (2024). URL https://doi.org/10.1186/s13100-024-00323-y.

[102] Li, H., Feng, X. & Chu, C. The design and construction of reference pangenome graphs with minigraph. *Genome Biology* **21**, 265 (2020). URL https://doi.org/10.1186/s13059-020-02168-z.

[103] Li, H. Minigraph 0.17 release (2022). URL https://github.com/lh3/minigraph/releases/tag/v0.17.

[104] Li, H. Minigraph documentation (2020). URL https://github.com/lh3/minigraph/blob/2f569ebe3071fcb242f1bf0eabbb917941b47239/tex/minigraph.tex.

[105] Li, H. Minigraph issue no. 27 (2020). URL https://github.com/lh3/minigraph/issues/27.

[106] Garrison, E. Pggb release v0.1 (2021). URL https://github.com/pangenome/pggb/tree/v0.1.0.

[107] Garrison, E. *et al.* Building pangenome graphs. *Nature Methods* **21**, 2008–2012 (2024). URL https://doi.org/10.1038/s41592-024-02430-3.

[108] Garrison, E. Graphical pangenomics [doctoral thesis] (2019).

[109] Marco-Sola, S., Moure, J. C., Moreto, M. & Espinosa, A. Fast gap-affine pairwise alignment using the wavefront algorithm. *Bioinformatics* **37**, 456–463 (2020). URL https://doi.org/10.1093/bioinformatics/btaa777. https://academic.oup.com/bioinformatics/article-pdf/37/4/456/50359789/btaa777.pdf.

[110] Kille, B., Garrison, E., Treangen, T. J. & Phillippy, A. M. Minmers are a generalization of minimizers that enable unbiased local jaccard estimation. *Bioinformatics* **39**, btad512 (2023). URL https://doi.org/10.1093/bioinformatics/btad512. https://academic.oup.com/bioinformatics/article-pdf/39/9/btad512/51648656/btad512.pdf.

[111] Garrison, E. & Guarracino, A. Unbiased pangenome graphs. *Bioinformatics* **39**, btac743 (2022). URL https://doi.org/10.1093/bioinformatics/btac743. https://academic.oup.com/bioinformatics/article-pdf/39/1/btac743/48448986/btac743.pdf.

[112] Dörr, D. Gfafix github (2024). URL https://github.com/marschall-lab/GFAffix.

[113] Cheng, L. *et al.* Leveraging a phased pangenome for haplotype design of hybrid potato. *Nature* **640**, 408–417 (2025). URL https://doi.org/10.1038/s41586-024-08476-9.

[114] Hickey, G. *et al.* Genotyping structural variants in pangenome graphs using the vg toolkit. *Genome Biology* **21**, 35 (2020). URL https://doi.org/10.1186/s13059-020-1941-7.

[115] Wick, R. R., Schultz, M. B., Zobel, J. & Holt, K. E. Bandage: interactive visualization of de novo genome assemblies. *Bioinformatics* **31**, 3350–3352 (2015). URL https://doi.org/10.1093/bioinformatics/btv383. https://academic.oup.com/bioinformatics/article-pdf/31/20/3350/49036207/bioinformatics_31_20_3350.pdf.

[116] Korobeynikov, A. Bandageng github (2022). URL https://github.com/marschall-lab/GFAffix.

[117] Gonnella, G., Niehus, N. & Kurtz, S. Gfaviz: flexible and interactive visualization of gfa sequence graphs. *Bioinformatics* **35**, 2853–2855 (2018). URL https://doi.org/10.1093/bioinformatics/bty1046. https://academic.oup.com/bioinformatics/article-pdf/35/16/2853/50719178/bty1046.pdf.

[118] Guarracino, A., Heumos, S., Nahnsen, S., Prins, P. & Garrison, E. Odgi: understanding pangenome graphs. *Bioinformatics* **38**, 3319–3326 (2022). URL https://doi.org/10.1093/bioinformatics/btac308. https://academic.oup.com/bioinformatics/article-pdf/38/13/3319/49883779/btac308.pdf.

[119] Sirén, J. *et al.* Personalized pangenome references. *Nature Methods* **21**, 2017–2023 (2024). URL https://doi.org/10.1038/s41592-024-02407-2.

[120] Milia, S. *et al.* Taurine pangenome uncovers a segmental duplication upstream of KIT associated with depigmentation in white-headed cattle. *Genome Res.* **35**, 1041–1052 (2025).

[121] Uffelmann, E. *et al.* Genome-wide association studies. *Nature Reviews Methods Primers* **1**, 59 (2021). URL https://doi.org/10.1038/s43586-021-00056-9.

[122] Tempel, S. *Using and Understanding RepeatMasker*, 29–51 (Humana Press, Totowa, NJ, 2012). URL https://doi.org/10.1007/978-1-61779-603-6_2.

[123] Goel, M., Sun, H., Jiao, W.-B. & Schneeberger, K. Syri: finding genomic rearrangements and local sequence differences from whole-genome assemblies. *Genome Biology* **20**, 277 (2019). URL https://doi.org/10.1186/s13059-019-1911-0.

[124] Goel, M. & Schneeberger, K. plotsr: visualizing structural similarities and rearrangements between multiple genomes. *Bioinformatics* **38**, 2922–2926 (2022). URL https://doi.org/10.1093/bioinformatics/btac196. https://academic.oup.com/bioinformatics/article-pdf/38/10/2922/46500909/btac196.pdf.

[125] Bradshaw, J. E. A brief history of the impact of potato genetics on the breeding of tetraploid potato cultivars for tuber propagation. *Potato Research* **65**, 461–501 (2022). URL https://doi.org/10.1007/s11540-021-09517-w.

[126] Schell, T., Greve, C. & Podsiadlowski, L. Establishing genome sequencing and assembly for non-model and emerging model organisms: a brief guide. *Frontiers in Zoology* **22**, 7 (2025). URL https://doi.org/10.1186/s12983-025-00561-7.

[127] Lander, E. S. & Waterman, M. S. Genomic mapping by fingerprinting random clones: A mathematical analysis. *Genomics* **2**, 231–239 (1988). URL https://www.sciencedirect.com/science/article/pii/0888754388900079.

[128] Illumina Inc. Hprc data release 2 (2014). URL https://www.illumina.com/documents/products/technotes/technote_coverage_calculation.pdf.

[129] Mitzenmacher, M. & Upfal, E. *Probability and computing : randomization and probabilistic techniques in algorithms and data analysis* (Cambridge University Press, Cambridge, United Kingdom, 2017), second edition edn.

[130] Sun, H. *et al.* Chromosome-scale and haplotype-resolved genome assembly of a tetraploid potato cultivar. *Nature Genetics* **54**, 342–348 (2022). URL https://doi.org/10.1038/s41588-022-01015-0.

[131] Xiao, P.-X. *et al.* Haplotype-resolved and near telomere-to-telomere assembly of the autotetraploid potato genome. *Genome Biology* **27**, 70 (2026). URL https://doi.org/10.1186/s13059-026-03980-9.

[132] Jiao, W.-B. & Schneeberger, K. Chromosome-level assemblies of multiple arabidopsis genomes reveal hotspots of rearrangements with altered evolutionary dynamics. *Nature Communications* **11**, 989 (2020). URL https://doi.org/10.1038/s41467-020-14779-y.

[133] Huang, X. *et al.* Analysis of natural allelic variation in *Arabidopsis* using a multiparent recombinant inbred line population. *Proceedings of the National Academy of Sciences* **108**, 4488–4493 (2011). URL https://www.pnas.org/doi/abs/10.1073/pnas.1100465108.

[134] HPRC. Pansn github (2022). URL https://github.com/pangenome/PanSN-spec.

[135] Alonge, M. *et al.* Automated assembly scaffolding elevates a new tomato system for high-throughput genome editing. *bioRxiv* (2021). URL https://www.biorxiv.org/content/early/2021/11/19/2021.11.18.469135. https://www.biorxiv.org/content/early/2021/11/19/2021.11.18.469135.full.pdf.

[136] Mohammad-Noori, M., Ghareghani, N. & Ghandi, M. Generalized gapped k-mer filters for robust frequency estimation. *Bulletin of the Iranian Mathematical Society* **50**, 68 (2024). URL https://doi.org/10.1007/s41980-024-00901-z.

[137] Zentgraf, J. & Rahmann, S. Design of Worst-Case-Optimal Spaced Seeds. In Brejová, B. & Patro, R. (eds.) *25th International Conference on Algorithms for Bioinformatics (WABI 2025)*, vol. 344 of *Leibniz International Proceedings in Informatics (LIPIcs)*, 22:1–22:17 (Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 2025). URL https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.WABI.2025.22.

[138] McGinty, S. P. *et al.* Cifi: accurate long-read chromosome conformation capture with low-input requirements. *Nature Communications* **17**, 215 (2025). URL https://doi.org/10.1038/s41467-025-66918-y.

[139] Deshpande, A. S. *et al.* Identifying synergistic high-order 3d chromatin conformations from genome-scale nanopore concatemer sequencing. *Nature Biotechnology* **40**, 1488–1499 (2022). URL https://doi.org/10.1038/s41587-022-01289-z.

[140] Remmert, M., Biegert, A., Hauser, A. & Söding, J. Hhblits: lightning-fast iterative protein sequence searching by hmm-hmm alignment. *Nature Methods* **9**, 173–175 (2012). URL https://doi.org/10.1038/nmeth.1818.

[141] Armstrong, J. *et al.* Progressive cactus is a multiple-genome aligner for the thousand-genome era. *Nature* **587**, 246–251 (2020). URL https://doi.org/10.1038/s41586-020-2871-y.

[142] Mistry, J. *et al.* Pfam: The protein families database in 2021. *Nucleic Acids Research* **49**, D412–D419 (2020). URL https://doi.org/10.1093/nar/gkaa913. https://academic.oup.com/nar/article-pdf/49/D1/D412/35363969/gkaa913.pdf.

[143] Li, H. Minigraph issue no. 155 (2018). URL https://github.com/lh3/minimap2/issues/155.

[144] Daily, J. Parasail: Simd c library for global, semi-global, and local pairwise sequence alignments. *BMC Bioinformatics* **17**, 81 (2016). URL https://doi.org/10.1186/s12859-016-0930-z.

[145] Kirsche, M. *et al.* Jasmine: Population-scale structural variant comparison and analysis. *bioRxiv* (2021). URL https://www.biorxiv.org/content/early/2021/05/28/2021.05.27.445886. https://www.biorxiv.org/content/early/2021/05/28/2021.05.27.445886.full.pdf.

[146] Dobzhansky, T. Studies on hybrid sterility. *Zeitschrift für Zellforschung und Mikroskopische Anatomie* **21**, 169–223 (1934). URL https://doi.org/10.1007/BF00374056.

[147] Orr, H. A. Dobzhansky, bateson, and the genetics of speciation. *Genetics* **144**, 1331–1335 (1996). URL https://doi.org/10.1093/genetics/144.4.1331. https://academic.oup.com/genetics/article-pdf/144/4/1331/34668837/genetics1331.pdf.

[148] Knief, U. *et al.* Evolution of chromosomal inversions across an avian radiation. *Molecular Biology and Evolution* **41**, msae092 (2024). URL https://doi.org/10.1093/molbev/msae092. https://academic.oup.com/mbe/article-pdf/41/6/msae092/58348219/msae092.pdf.

Appendix

Phasing Requirements

```
#!/bin/env Rscript

library(zipfR)

to_pr <- function(x) {
    -log10(x)/10
}

from_pr <- function(x) {
    10**(-x/10)
}

hapcov <- function(n=2, PR=20){
    (n-1)*(log(n)-log(from_pr(PR))) + 1
}

totalcov <- function(c, PR=20){
    Rgamma.inv(c+1, 1-from_pr(PR))
}

len_correction <- function(c, i, l) {
    if(i >= l) {
        message("IBD regions longer than read length! Not phaseable")
    }
    c*l/(l-i)
}

calc_total <- function(i, l, n=2, PR=20) {
    len_correction(
                        totalcov(
                                    hapcov(n=n, PR=PR),
                                    PR=PR),
                        i=i, l=l)
}

## run script
args = commandArgs(trailingOnly=TRUE)
#print(args)

n <- 2
ibd <- 10000 # length of IBD blocks
PR <- 20 # alpha, in units of PR

readlen <- 200000 # read length. TODO replace with empirical read len distribution
    ?
```

```
45
46 if (length(args) == 0) {
47     message("Welcome to the polyploid phasing coverage calculator!
48 Usage: covcalc.R [ploidy] [desired min. phasing PR] [max. len of IBD regions] [min
       . read len to consider]
49 Default vals: 2, 20, 100,000, 200,000
50
51 The calculator will print the estimate for the minimum required coverage meeting
       the desired phasing quality, as well as a breakdown of what is contributing to
       this estimated minimum.")
52     quit()
53 } else if (length(args) < 4) {
54     message("Not enough parameters! Using defaults: ", n, ", ", PR, ", ", ibd, ",
           ", readlen)
55 } else {
56     n <- as.numeric(args[1])
57     PR <- as.numeric(args[2])
58     ibd <- as.numeric(args[3])
59     readlen <- as.numeric(args[4])
60 }
61
62 #req <- calc_total(i=ibd, l=readlen, n=n, PR=PR)
63 req_hapcov <- hapcov(n=n, PR=PR)
64 req_totcov <- totalcov(req_hapcov, PR=PR)
65 lencorr <- len_correction(req_totcov, i=ibd, l=readlen)
66 message(round(lencorr, 1), "x required overall after length correction.")
67 message(round(req_totcov, 1), "x required to achieve ", round(req_hapcov, 1), "x
       at all positions @ PR ", PR)
68 message(round(req_hapcov, 1), "x required to cover all ", n, " haplotypes @ PR ",
       PR)
69
70 yield <- 20e9
71 genome_len <- 0.6e9
72 req_flowcells <- genome_len*lencorr/yield
73 message("This is estimated to require ", round(req_flowcells, 1), " flowcells per
       sample @ ", yield/1e9, " Gbp of long reads per flowcell and a genome size of ",
        genome_len/1e9, " Gbp")
74 message("All coverages refer to haploid genome size!")
75
76 quit()
77
78 ###############
79 ## PLOTTING ##
80 ###############
81 # Code used for plotting, pasted here for reference
82
83 library(ggplot2)
84
85 # ggplot does not like if statements, apparently
86 len_correction <- function(c, i, l) {
87     #if(i >= l) {
88     #   message("IBD regions longer than read length! Not phaseable")
89     #}
90     c*l/(l-i)
91 }
92
93 # Plot by IBD size
94
95 window <- ggplot() +
96     xlim(0, 100) + ylim(0, 200) +
```

```
    xlab("Shared region size [kbp]") + ylab("Req, coverage") +
    labs(colour="ploidy", linetype="Read length")

window +
    geom_function(fun = {function(x) calc_total(x, 100, n=2, PR=20)}
                        , aes(colour="diploid", linetype="100 kbp")) +
    geom_function(fun = {function(x) calc_total(x, 100, n=3, PR=20)}
                        , aes(colour="triploid", linetype="100 kbp")) +
    geom_function(fun = {function(x) calc_total(x, 100, n=4, PR=20)}
                        , aes(colour="tetraploid", linetype="100 kbp")) +
    geom_function(fun = {function(x) calc_total(x, 100, n=8, PR=20)}
                        , aes(colour="octoploid", linetype="100 kbp")) +
    # longer reads
    geom_function(fun = {function(x) calc_total(x, 200, n=2, PR=20)}
                        , aes(colour="diploid", linetype="200 kbp")) +
    geom_function(fun = {function(x) calc_total(x, 200, n=3, PR=20)}
                        , aes(colour="triploid", linetype="200 kbp")) +
    geom_function(fun = {function(x) calc_total(x, 200, n=4, PR=20)}
                        , aes(colour="tetraploid", linetype="200 kbp")) +
    geom_function(fun = {function(x) calc_total(x, 200, n=8, PR=20)}
                        , aes(colour="octoploid", linetype="200 kbp"))
#    # lower PR
#    geom_function(fun = {function(x) calc_total(x, 100, n=2, PR=10)}
#                        , aes(colour="diploid, PR 10", linetype="100 kbp reads")) +
#    geom_function(fun = {function(x) calc_total(x, 100, n=3, PR=10)}
#                        , aes(colour="triploid, PR 10", linetype="100 kbp reads")) +
#    geom_function(fun = {function(x) calc_total(x, 100, n=4, PR=10)}
#                        , aes(colour="tetraploid, PR 10", linetype="100 kbp reads"))
     +
#    geom_function(fun = {function(x) calc_total(x, 100, n=8, PR=10)}
#                        , aes(colour="octoploid, PR 10", linetype="100 kbp reads"))
    +

ggsave("covbyibd.pdf", width=10, height=3)

window <- ggplot() +
    xlim(1.3, 5) + ylim(0, 200) +
    xlab("Read len/Shared region size") + ylab("Req, coverage") +
    labs(colour="ploidy", linetype="Quality")
    #+ scale_y_log10()

window +
    geom_function(fun = {function(x) calc_total(1, x, n=2, PR=30)}
                        , aes(colour="diploid", linetype="PR 30")) +
    geom_function(fun = {function(x) calc_total(1, x, n=3, PR=30)}
                        , aes(colour="triploid", linetype="PR 30")) +
    geom_function(fun = {function(x) calc_total(1, x, n=4, PR=30)}
                        , aes(colour="tetraploid", linetype="PR 30")) +
    geom_function(fun = {function(x) calc_total(1, x, n=8, PR=30)}
                        , aes(colour="octoploid", linetype="PR 30")) +
    geom_function(fun = {function(x) calc_total(1, x, n=2, PR=20)}
                        , aes(colour="diploid", linetype="PR 20")) +
    geom_function(fun = {function(x) calc_total(1, x, n=3, PR=20)}
                        , aes(colour="triploid", linetype="PR 20")) +
    geom_function(fun = {function(x) calc_total(1, x, n=4, PR=20)}
                        , aes(colour="tetraploid", linetype="PR 20")) +
    geom_function(fun = {function(x) calc_total(1, x, n=8, PR=20)}
                        , aes(colour="octoploid", linetype="PR 20"))
#    geom_function(fun = {function(x) calc_total(1, x, n=2, PR=15)}
#                        , aes(colour="diploid", linetype="PR 15")) +
```

```
#   geom_function(fun = {function(x) calc_total(1, x, n=3, PR=15)}
#                     , aes(colour="triploid", linetype="PR 15")) +
#   geom_function(fun = {function(x) calc_total(1, x, n=4, PR=15)}
#                     , aes(colour="tetraploid", linetype="PR 15")) +
#   geom_function(fun = {function(x) calc_total(1, x, n=8, PR=15)}
#                     , aes(colour="octoploid", linetype="PR 15"))

ggsave("covbyadjrlen.pdf", width=10, height=3)
```

Dataset Processing

msyd_preprocessing.nf

```
#!/bin/env nextflow

//nextflow.preview.output=true

process align {
    memory 128.GB
    cpus 4
    time 2.hour

    input:
        tuple val(sample), path(genome)
        path ref

    output:
        tuple val(sample), path(genome), path("*.paf", arity: 1)

    script:
    """
    minimap2 -t ${task.cpus} -cx asm20 --eqx ${ref} ${genome} > ${sample}.paf
    """
}

process syri {
    memory 64.GB // this doesn't actually need so much mem, but non-scheduled jobs keep killing mine...
    cpus 8
    time 5.hour

    input:
        tuple val(sample), path(genome), path(aln)
        path ref

    output:
        tuple val(sample), path(genome), path(aln), path("*syri.out", arity: 1), path("*syri.vcf", arity: 1), path("*syri.log", arity: 1)
    script:
    """
    syri --nc 8 -F P --cigar --prefix ${sample} --samplename ${sample} -c ${aln} -r ${ref} -q ${genome} --lf ${sample}.syri.log
    rm ${sample}.syri.log # only keep if something fails in the syri call
    """
}


params.ref = file('./ref.fa.bgz')
params.samplesheet = file('./samplesheet.csv')

workflow {

    main:
        samplesheet = channel.fromPath(params.samplesheet).splitCsv()
        samplesheet = samplesheet.map{row -> tuple(row[0], file(row[1]))}
        samplesheet.view { row -> "${row[0]} - ${row[1]}" }
        samplenames = samplesheet.map{row -> row[0]}
        genomes = samplesheet.map{row -> file(row[1])}
```

```
        aligned = align(samplesheet, file(params.ref))
        syris = syri(aligned, file(params.ref))
        // do in a script instead, nextflow is too messy
        //syris.collectFile(name: 'full.tsv', newLine: true)
        //  .subscribe { file ->
        //      println "Entries are saved to file: $file"
        //      println "File content is: ${file.text}"
        //}

    publish:
        syriouts = syris

}

output {
    syriouts {
        mode 'copy'
    }
}
```

cleanup_nf.sh

```
#!/bin/bash

dir=$1

mvcmd='cp' # 'mv' if space is an issue

## init folder structure
mkdir -p dataset
mkdir -p dataset/seq
mkdir -p dataset/aln
mkdir -p dataset/syri
mkdir -p dataset/vcf
echo "#name aln syri    vcf seq" > dataset/full.tsv

## move files, per sample
samples=$(ls $dir/*.fa.bgz)
for sample in $samples; do
    name=$(basename $sample .fa.bgz)

    # move files
    $mvcmd $dir/$name.fa.bgz dataset/seq/
    $mvcmd $dir/$name.paf dataset/aln/
    $mvcmd $dir/${name}syri.out dataset/syri
    $mvcmd $dir/${name}syri.vcf dataset/vcf

    # add to .tsv
    echo "$name dataset/aln/$name.paf   dataset/syri/${name}syri.out    dataset/
       vcf/${name}syri.vcf dataset/seq/$name.fa.bgz" >> dataset/full.tsv

done

cut -f 1,5 dataset/full.tsv > dataset/fastas.tsv
```